

PP_zsmf_t1

Disclaimer

Zur Erstellung dieses Dokuments wurde sehr viel mit LLMs gearbeitet. Die Informationen können fehlerhaft und unvollständig sein, deshalb unbedingt schon mit einer Grundahnung an die Sache herangehen und nicht ausschließlich die Zusammenfassung zum Lernen, sondern viel eher zum schnellen Wiederholen verwenden!

Copyright

Da ich keine Schwierigkeiten in Bezug auf Copyright bekommen will, sind Beispiele / Darstellungen, welche ich normalerweise als Screenshot einfügen würde, im finalen Export nur Referenzen. Sie sollten trotzdem anklickbar sein und dich auf die TUWEL Seite der Slide führen, und auch die Seite öffnen, die ich einbinden wollte.

Inhalt:

- [vo1](#)
- [vo2](#)
- [vo3](#)
- [vo4](#)
- [vo5](#)
- [vo6](#)

Berechnungsmodelle als formale Grundlagen

- **Berechnungsmodell:** Ein formales Modell, das alles Berechenbare ausdrücken kann.
 - Muss die **Mächtigkeit der Turing-Maschine** besitzen (d.h. eine "vollständige Sprache" sein).
 - Muss sich auch praktisch (ohne übermäßigen Aufwand) in Programmiersprachen und Werkzeuge umsetzen lassen.
- **Beispiele** für formale Grundlagen:
 - **Funktionen:** z.B. primitiv rekursive, **μ -rekursive Funktionen** oder der **λ -Kalkül**.
 - **Logik:** Prädikatenlogik, Horn-Klauseln (Basis für Prolog).
 - **Zeitabhängigkeit:** Temporale Logiken, Petri-Netze (z.B. für Nebenläufigkeit).
 - **Strukturen:** Freie Algebren (wichtig für Module, Typen, Spezifikationen).
 - **Nebenläufigkeit:** Prozesskalküle (z.B. CSP, π -Kalkül).
 - **Grammatiken:** Automaten (z.B. Turing-Maschine).
 - **Imperative Konstrukte:** LOOP-, WHILE-, GOTO-Sprachen, PRAM.

Der λ -Kalkül (Lambda-Kalkül)

λ -Ausdrücke, freie Variablen, Ersetzung

- Gegeben: Unendliche Menge V an **Variablen** (beinhaltet auch Konstanten und Literale).
- **Syntax der λ -Ausdrücke (Menge E):**
 1. **Variable:** $v \in E$ (wenn $v \in V$).
 2. **Anwendung:** $f e \in E$ (wenn $e, f \in E$). (Funktion f wird auf Argument e angewendet).
 3. **Abstraktion:** $\lambda v. e \in E$ (wenn $v \in V; e \in E$). (Definition einer Funktion mit Parameter v und Rumpf/Ergebnis e).
- **Freie Variablen $fv(e)$:** Variablen, die nicht durch ein λ als Parameter **gebunden** sind.
 - $fv(v) = \{v\}$.
 - $fv(fe) = fv(f) \cup fv(e)$.
 - $fv(\lambda v. e) = fv(e) \setminus \{v\}$ (Hier ist v gebunden und somit nicht mehr frei).
- **Ersetzung $[e/v]f$:**
 - Ersetzt alle **freien** Vorkommen von v in f durch e .
 - $[e/v]v = e$
 - $[e/v]u = u$ (wenn $u \neq v$)
 - $[e/v](fg) = ([e/v]f)([e/v]g)$
 - $[e/v](\lambda v. f) = \lambda v. f$ (Das v in $\lambda v. f$ ist gebunden und wird nicht ersetzt).
 - $[e/v](\lambda u. f) = \lambda u. [e/v]f$ (Bedingung: $u \neq v$ **und** $u \notin fv(e)$, um Namenskonflikte zu vermeiden).

Regeln des λ -Kalküls

- **Äquivalenzregeln ($\equiv$):** Beschreiben, wann zwei Ausdrücke gleichwertig sind (sind ungerichtet, d.h. gelten in beide Richtungen).
 - **α -Konversion (Umbenennung):** $\lambda u. e \equiv \lambda v. [v/u]e$ (wenn $v \notin fv(e)$). (Parameter dürfen umbenannt werden).
 - **β -Konversion (Anwendung/Parameterübergabe):** $(\lambda v. f)e \equiv [e/v]f$. (Die wichtigste Regel: Führt die Funktionsanwendung aus).
 - **η -Konversion (Extensionalität/Optimierung):** $\lambda v. (e v) \equiv e$ (wenn $v \notin fv(e)$). (Entfernt überflüssige Abstraktionen).
- **Reduktionsregeln ($\rightarrow$):**
 - Gerichtete Version der Konversionsregeln (immer von links nach rechts) zur Vereinfachung (Berechnung).
 - **β -Reduktion:** $(\lambda v. f)e \rightarrow [e/v]f$.
 - **η -Reduktion:** $\lambda v. (e v) \rightarrow e$ (wenn $v \notin fv(e)$).
- **Berechnung & Normalform:**
 - Eine **Berechnung** ist die (wiederholte) Reduktion eines Ausdrucks bis zu seiner **Normalform**.
 - Ein Ausdruck ist in **Normalform**, wenn keine Reduktionsregeln (β oder η) mehr anwendbar sind. (Man verwendet α -Konversion, um Reduktionen zu ermöglichen).

Eigenschaften des λ -Kalküls

- **Terminierung:** Nicht jeder λ -Ausdruck ist zu einer Normalform reduzierbar.
 - Es gibt **Endlosreduktionen** (z.B. der Ω -Kombinator: $(\lambda v. (v v))(\lambda v. (v v))$). Dies ist aufgrund des Halteproblems der Turing-Maschine unvermeidlich.
- **Konfluenz (Church-Rosser-Eigenschaft):**
 - Die **Reihenfolge der Reduktionen** ist entscheidend. Selbst wenn eine Normalform existiert, kann man durch "falsche" Reduktionsschritte (z.B. innere zuerst, "Eager Evaluation") in einer Endlosschleife landen.
 - **Sichere Strategie:** Immer den **äußersten** reduzierbaren Ausdruck wählen (entspricht "Lazy Evaluation").
 - **Eindeutigkeit:** Jede Reduktionsreihenfolge, die terminiert, führt (bis auf α -Konversion) zur **gleichen Normalform**. Das Ergebnis ist also eindeutig.
- **Mächtigkeit:** Der λ -Kalkül ist extrem einfach (nur Funktionen), aber **so mächtig wie die Turing-Maschine**.
- Die Einfachheit des Kalküls bedeutet nicht, dass Programme darin einfach zu schreiben oder zu verstehen sind.

Beispiele für Reduktionen im λ -Kalkül

- **Endlosreduktion:**

$$(\lambda v. (v v))(\lambda v. (v v)) \rightarrow (\lambda v. (v v))(\lambda v. (v v)) \rightarrow \dots$$

- **Mehrere Parameter (Currying):** Eine Funktion (mit Parameter u) gibt eine neue Funktion (mit Parameter v) zurück.

$$((\lambda u. (\lambda v. ((u\ v)(v\ u))))a)b \rightarrow (\lambda v. ((a\ v)(v\ a)))b \rightarrow (a\ b)(b\ a)$$

- **Bedingte Ausführung (Boolesche Logik):**

- `true` = $\lambda u. (\lambda v. u)$ (Nimmt zwei Argumente, gibt das erste zurück).
- `false` = $\lambda u. (\lambda v. v)$ (Nimmt zwei Argumente, gibt das zweite zurück).
- `if_then_else` = $\lambda b. (\lambda u. (\lambda v. ((b\ u)v)))$ (Nimmt b (true/false), dann u (then-Zweig), dann v (else-Zweig)).

- **Beispiel-Reduktion (if true a b):**

- $((((\lambda b. (\lambda u. (\lambda v. ((b\ u)v)))))(\lambda u. (\lambda v. u)))a)b$
- $\rightarrow ((\lambda u. (\lambda v. (((\lambda u. (\lambda v. u))u)v)))a)b$ (Äußeres b wird durch `true` ersetzt)
- $\rightarrow (\lambda v. (((\lambda u. (\lambda v. u))a)v))b$ (Äußeres u wird durch a ersetzt)
- $\rightarrow ((\lambda u. (\lambda v. u))a)b$ (Äußeres v wird durch b ersetzt)
- $\rightarrow (\lambda v. a)b$ (Inneres u wird durch a ersetzt)
- $\rightarrow a$ (Inneres v wird durch b ersetzt, b wird verworfen)

Anforderungen an die praktische Realisierung von Berechnungsmodellen

(Was macht ein Paradigma erfolgreich?)

- **Kombinierbarkeit:** Bestehende Programmtteile müssen sich einfach und (wichtig!) **skalierbar** zu größeren Einheiten kombinieren lassen.
- **Konsistenz:** Die verschiedenen Konzepte einer Sprache (z.B. für Algorithmen und Datenstrukturen) müssen widerspruchsfrei zusammenpassen.
 - *Praxis-Kompromiss:* Konsistenz ist oft wichtiger als die "optimale" Grundlage für jeden einzelnen Aspekt.
- **Abstraktion:** Wichtigstes Ziel ist die Abstraktion von Hardware- und Systemdetails, um **Portabilität** (Lauffähigkeit auf verschiedenen Systemen) zu erreichen.
- **Systemnähe:** Programme müssen effizient auf realer Hardware laufen und diese bei Bedarf (z.B. für Treiber) direkt ansprechen können.
- **Unterstützung:** Ein Paradigma braucht verfügbare Programmiersprachen, Werkzeuge, Bibliotheken und oft auch die Unterstützung einflussreicher Unternehmen (z.B. Java, C#).
- **Beharrungsvermögen:** Paradigmenwechsel werden von Entwicklern nur sehr langsam angenommen, da Wissen verloren geht und neue Erfahrungen nötig sind.
 - Ein Wechsel erfordert oft eine **"Killerapplikation"**, die den überlegenen Nutzen des neuen Paradigmas unübersehbar beweist.

Evolution und feine Programmstrukturen

Evolution auf Basis von Widersprüchen

- **Der Kernwiderspruch:** Programme sollen gleichzeitig 3 Ziele erfüllen, die sich widersprechen:
 1. **Flexibilität und Ausdruckskraft** (Alles darstellen können).
 2. **Lesbarkeit und Sicherheit** (Absichten leicht erkennen, Fehler vermeiden).
 3. **Einfache Verständlichkeit** (Klarheit, was die Konzepte bewirken).
- **Der Kompromiss:**
 - Früher: Dynamisch typisiert (Fokus auf 1) vs. Statisch typisiert (Fokus auf 2).
 - Heute: Moderne Sprachen schaffen oft 1+2 (Flexibilität + Sicherheit), aber meist **auf Kosten der Einfachheit (3)**.
- **Evolution statt Revolution:**
 - Dieses Spannungsfeld (auch "Modeerscheinungen" wie statisch vs. dynamisch) ist die Triebfeder für die **Evolution** von Paradigmen.
 - **Wie ändern sich Paradigmen?**
 1. Entwickler sammeln neue Erfahrungen → persönliche Programmierstile ändern sich.
 2. Die **Notwendigkeit zur Zusammenarbeit** an Projekten erzwingt eine Anpassung und Vereinheitlichung der Stile.
 3. Dadurch ändert sich das gesamte Paradigma langsam und evolutionär, ohne starke Brüche oder Aufspaltung.

Prozeduren und Strukturierte Programmierung

- **Prozedur:** Ähnlich einer Funktion, aber der Schwerpunkt liegt auf **Seiteneffekten** (destruktive Zuweisungen), die den globalen Programmzustand ändern.
- **Strukturierte Programmierung:**
 - Ein Programmierstil, bei dem Prozeduren **ausschließlich** aus drei Kontrollstrukturen aufgebaut sind:
 1. **Sequenz** (Schritt für Schritt).
 2. **Auswahl** (Verzweigung, z.B. `if`, `switch`).
 3. **Wiederholung** (Schleife oder Rekursion).
 - **Vorteil:** Diese 3 Konstrukte sind als **Denkmuster** hinreichend, um jedes Programmverhalten auszudrücken (Turing-vollständig).
 - Jede Struktur hat nur **einen Einstiegs- und einen Ausstiegspunkt**. Dies macht den Kontrollfluss im Code visuell leicht nachvollziehbar (z.B. durch Einrückung).
- **Gefahren (Anti-Muster):**
 - `goto` ist das Gegenteil; es erlaubt beliebige Sprünge und gilt als extrem fehleranfällig, da es die strukturierten Denkmuster bricht.
 - Auch `break`, `continue` oder `throw` (Ausnahmen) durchbrechen diese Denkmuster, da sie "unsichtbare" zusätzliche Ausgänge schaffen.
- Die Grundidee der strukturierten Programmierung wurde in fast jedem modernen Paradigma übernommen.

Funktionen als Daten (Funktionale Programmierung)

- **Mächtigkeit und Vollständigkeit:**
 - *Primitiv rekursive Funktionen* (die nur elementare Daten, aber keine Funktionen verarbeiten) sind **nicht Turing-vollständig**.
 - Vollständigkeit (Turing-Mächtigkeit) kann erreicht werden durch:
 1. Zusätzliche Kontrollstrukturen (z.B. eine `while`-Schleife / μ -Operator).
 2. **Funktionen als Daten** behandeln (wie im λ -Kalkül). (D.h. Funktionen zur Laufzeit erzeugen, übergeben, speichern, zurückgeben).
- **Konsequenzen von "Funktionen als Daten":**
 - **Vorteil:** Enorme Ausdrucksstärke und Flexibilität.
 - **Nachteil:** Der **Kontrollfluss wird verschleiert**, da Daten (die übergebenen Funktionen) nun selbst Kontrollfluss enthalten.
- **Lösung (Applikative Programmierung):**
 - Man nutzt Funktionen als Daten, um **Funktionen höherer Ordnung** (funktionale Formen, z.B. `map`, `filter`) zu bauen.
 - Diese ersetzen die imperativen Kontrollstrukturen (wie `for`-Schleifen) und passen oft gut zu den Denkmustern der strukturierten Programmierung.
- **Abwägung:** Freiheit/Ausdrucksstärke (funktional) vs. einfache, überschaubare Denkmuster (prozedural).

Objekte als Daten (Objektorientierte Programmierung)

- **Parallele Argumentation:** Ähnlich wie bei "Funktionen als Daten", wird die Objektorientierung erst durch "Objekte als Daten" (zur Laufzeit erzeugen, übergeben, speichern) mächtig.
- **Der fundamentale Unterschied:**
 - Die Denkmuster im OO-Paradigma basieren auf **Abstraktionen des Objektverhaltens** (z.B. Kapselung, Polymorphie, Ersetzbarkeit).
 - Die Denkmuster im Funktionalen Paradigma basieren auf **Funktionalen Formen** (z.B. `map`, `reduce`).
 - → Objektorientierte und funktionale Denkmuster sind fundamental verschieden.
- **Kombination (z.B. in Java):**
 - Objekte sind primär "Daten".
 - Funktionen (Lambdas) können *indirekt* als Daten behandelt werden, indem sie in spezielle Objekte (sog. "funktionale Interfaces") verpackt werden.
 - Die Paradigmen können **koexistieren**, aber wegen inhärenter Widersprüche (z.B. Seiteneffekte vs. Seiteneffektfreiheit) **nicht vollständig zu einem Paradigma verschmelzen**.
- **Abwägung der Paradigmen:**
 - **Objektorientiert:** Fokus auf Abstraktion.
 - **Funktional:** Fokus auf Ausdrucksstärke.
 - **Prozedural:** Fokus auf wenige, einfache Denkmuster.

Programmorganisation: Modularisierung

Modularisierung (auch **Faktorisierung** genannt) bezeichnet die Zerlegung von Programmen in relativ lose gekoppelte, austauschbare Einheiten.

Allgemeine Eigenschaften von Modularisierungseinheiten:

- Besitzen einen **separaten Namensraum** (vermeidet Namenskonflikte).
 - Setzen **Datenabstraktion** um (verstecken interne Details).
 - Haben eine definierte **Schnittstelle** (legen fest, was **exportiert**, d.h. nach außen sichtbar, ist).
-

Formen von Modularisierungseinheiten

- **Objekt:**
 - Existiert zur **Laufzeit**.
 - Ist **anonym** (hat keinen festen Namen, wird über Referenzen angesprochen).
 - Besitzt drei wesentliche Eigenschaften:
 1. **Identität** (ist einzigartig im Speicher, ==).
 2. **Zustand** (die Werte seiner internen Variablen).
 3. **Verhalten** (definiert durch seine Methoden).
- **Klasse:**
 - Dient als "Schablone" oder "Bauplan" für die **Objekterzeugung** (mittels Konstruktoren).
 - Dient der **Klassifizierung** von Objekten.
 - In Java: z.B. `class` oder `interface` (für nicht-statische Teile).
- **Modul:**
 - Ist eine **Übersetzungseinheit** (eine Datei, die einzeln kompiliert wird).
 - Importiert andere Module explizit über deren **Namen**.
 - Abhängigkeiten dürfen **zyklenfrei** sein (A importiert B, B darf nicht A importieren), um getrennte Übersetzung zu ermöglichen.
 - In Java: z.B. eine `class` (für statische Teile: `static` Methoden/Variablen).
- **Komponente:**
 - Ist eine **Übersetzungseinheit**.
 - Importiert benötigte Dienste **anonym** (weiß zur Übersetzungszeit noch nicht, wer den Dienst bereitstellt).
 - Die Verbindung (das "Befüllen der Lücken") geschieht erst beim **Deployment** (Zusammenbau der Anwendung, statisch oder zur Laufzeit).
 - In Java: z.B. eine Java-EE-Bean.
- **Namensraum (Package):**
 - Dient rein der **Organisation** und Gruppierung von anderen Modularisierungseinheiten.
 - Oft hierarchisch (wie Verzeichnisse).

- In Java: z.B. ein `package`.
-

Programmorganisation: Parametrisierung

Parametrisierung bedeutet, dass in Modularisierungseinheiten "Lücken" (Parameter) belassen werden, die erst zu einem späteren Zeitpunkt befüllt werden.

- **Parametrisierung von Objekten (zur Laufzeit):**
 - Lücken sind meist Objektvariablen, die initialisiert werden müssen.
 - Werte gelangen auf folgende Arten ins Objekt:
 - **Konstruktor:** Der übliche Weg; Werte werden bei der Erzeugung (`new`) übergeben.
 - **Initialisierungsmethode:** Eine separate Methode (z.B. `init(...)`), die nach der Erzeugung aufgerufen wird. Nötig z.B. bei zyklischen Abhängigkeiten (Objekt A braucht Objekt B und B braucht A) oder bei `clone()`.
 - **Zentrale Ablage:** Das Objekt holt sich seine Werte selbst von einem vorbestimmten, globalen Platz (z.B. einer statischen Variable oder einer Registry).
 - **Parametrisierung von Klassen, Modulen, Komponenten (zur Übersetzungs-/Deployment-Zeit):**
 - Hier sind die "Werte" oft keine normalen Daten, sondern Konzepte wie Typen oder Code.
 - **Generizität:** Lücken sind **Typparameter** (z.B. `<T>`), die bei der Verwendung durch konkrete Typen (z.B. `String`) ersetzt werden. (Erfolgt konzeptuell zur Übersetzungszeit).
 - **Annotationen:** Metadaten (`@Override`, `@Test`), die an Programmteile "angeheftet" werden. Sie werden von Werkzeugen (Compiler, Frameworks) zur Laufzeit oder Übersetzungszeit explizit ausgelesen und steuern das Verhalten.
 - **Aspekte (Aspektorientierte Programmierung):** Spezifizieren Modifikationen am Code von außen (z.B. "Führe diesen Code *vor* jedem Methodenaufruf aus"). Ein "Aspect-Weaver" ändert den Code (vor der Kompilierung oder zur Laufzeit).
 - **Problem:** Es besteht eine **starke Abhängigkeit** zwischen den definierten Lücken (Parametern) und den Werten/Typen, die zum Befüllen verwendet werden. Ändert man die Lücken (z.B. einen Parameter einer Methode), müssen *alle* Stellen, die diese Lücken befüllen (alle Aufrufe), ebenfalls geändert werden.
-

Programmorganisation: Ersetzbarkeit

- **Definition:** Eine Einheit *A* ist durch eine Einheit *B* ersetzbar, wenn der Ersatz von *A* durch *B* **keine Änderung** bei den Verwendern (Clients) von *A* verlangt.
- Ersetzbarkeit hängt nicht nur von *A* und *B* ab, sondern auch von den **Erwartungen** der Clients an *A*.
- **Voraussetzung:** Klar definierte **Schnittstellen**, welche die erlaubten Erwartungen beschreiben.
- Damit *B* *A* ersetzen kann, muss gelten:

- Die Schnittstelle von B **impliziert** die Schnittstelle von A .
 - Die Schnittstelle von B kann **mehr Details** festlegen (strenger sein) als die von A .
 - **Spezifikationsarten für Schnittstellen (und deren Erwartungen):**
 - **Signatur:** (z.B. Methodenname, Parameter-Typen, Rückgabebetyp).
 - **Vorteil:** Simpel, automatisch (vom Compiler) prüfbar.
 - **Nachteil:** Irrtum möglich, da die **Bedeutung (Semantik)** nicht festgelegt ist.
 - **Abstraktion realer Welt:** (z.B. "Auto" **is-a** "Fahrzeug").
 - **Vorteil:** Intuitiv.
 - **Nachteil:** Basiert auf Intuition und ist oft fehlerbehaftet (garantiert keine echte Ersetzbarkeit).
 - **Zusicherungen (Design-by-Contract, DbC):** (z.B. Vorbedingungen, Nachbedingungen, Invarianten).
 - **Vorteil:** Präzise Definition des "Vertrags" zwischen Client und Server.
 - **Nachteil:** Aufwändig zu spezifizieren und zu prüfen.
 - **Überprüfbare Protokolle:** (Formale Modelle des Interaktionsablaufs).
 - **Nachteil:** Sehr aufwändig, gelten als noch nicht praxistauglich.
-

Abstraktion

Das **Abstraktionsprinzip** (Vermeidung von Code-Duplikaten) greift oft zu kurz. Entscheidend ist die **Absicht** (die Abstraktion) hinter dem Code.

- **Unterschiedliche Semantik (Code), gleiche Abstraktion (Absicht):**
 - Zwei verschiedene Implementierungen einer `swap`-Methode (eine mit Hilfsvariable, eine mit XOR-Operationen).
 - Trotz unterschiedlichem Code ist die Abstraktion (definiert durch den Kommentar `// swap a[i] and a[j]; i != j`) dieselbe.
[pp25v02, p.5](#)
 - **Gleiche Semantik (Code), unterschiedliche Abstraktionen (Absicht):**
 - Eine Methode `compare(x, y)` und `subtract(x, y)`, die beide nur `return x - y;` implementieren.
 - Die Abstraktion `compare` (soll < 0 , 0 , oder > 0 liefern) ist fundamental anders als `subtract` (soll die Differenz liefern).
-

Arten von Abstraktionen (für Funktionen/Methoden)

- **λ -Abstraktion (Lambda-Abstraktion):**
 - Die Abstraktion **ist** die Implementierung (der Code-Rumpf), wie im λ -Kalkül.
 - Name und Kommentare sind bedeutungslos.
 - Jede inhaltliche Änderung der Implementierung ändert sofort die Abstraktion.

- **Strukturelle Abstraktion:**
 - Die Abstraktion *ist* nur die Signatur (Typen der Parameter und des Rückgabewerts).
 - Name, Kommentare und Implementierung sind bedeutungslos.
 - Wird z.B. für funktionale Interfaces (Lambdas) in Java verwendet.
 - **Nominale Abstraktion:**
 - Die Abstraktion wird durch den **Namen** und die **Kommentare** (Zusicherungen, Semantik) definiert.
 - Die Implementierung muss zu dieser Beschreibung passen (oder ist unbekannt, z.B. bei abstrakten Methoden).
 - Eine Änderung des Namens oder der Kommentare ändert die Abstraktion (selbst wenn der Code gleich bleibt).
 - Dies ist die Standard-Abstraktion in der Objektorientierung.
 - **Basisabstraktion:**
 - Eine von der Programmiersprache vorgegebene Bedeutung (z.B. `if`, `while`, `int`).
 - Funktioniert wie eine nominale Abstraktion, kann aber nicht geändert werden.
-

Datenabstraktion

- Hier ist die Abstraktionseinheit eine **Modularisierungseinheit** (z.B. eine Klasse).
 - Sie basiert auf zwei Prinzipien:
 1. **Datenkapselung:** Zusammenfassen von Daten (Variablen) und Funktionen (Methoden) zu einer untrennbaren Einheit.
 2. **Data-Hiding:** Beschränken der Sichtbarkeit, Trennung von interner Implementierung (z.B. `private`) und externer Schnittstelle (z.B. `public`).
 - **Abstrakter Datentyp (ADT):**
 - Ist die **Schnittstelle** einer Modularisierungseinheit.
 - **Arten von ADTs:**
 - **Strukturell:** Nur die Signatur zählt (strukturelle Abstraktion). Selten.
 - **Nominal:** Der Name und die Beschreibungen (Kommentare, Zusicherungen) sind wesentlich (nominale Abstraktion). Der Normalfall in OO.
 - **Algebraisch:** Die Beziehungen zwischen den Funktionen/Daten sind formal festgelegt (z.B. durch Axiome). Typisch für rein funktionale Sprachen.
-

Abstraktionshierarchien

Abstraktionen selbst sind oft vage (intuitive Vorstellungen), aber die *Beziehungen* zwischen den abstrakten Datentypen (Typen) müssen klar spezifiziert werden.

- **Arten von Hierarchien:**

- **Beziehung in der realen Welt:** Basiert auf vagen "is-a"-Beziehungen (z.B. "ein Auto *ist ein* Fahrzeug"). Dient oft als Startpunkt der objektorientierten Modellierung, garantiert aber **keine Ersetzbarkeit**.
 - **Untertypbeziehung (Subtyping):**
 - Definiert durch das **Ersetzbarkeitsprinzip**: B ist Untertyp von A , wenn jede Instanz von B überall dort verwendet werden kann, wo eine Instanz von A erwartet wird.
 - Dies ist **essenziell** für die objektorientierte Programmierung, da es Ersetzbarkeit garantiert.
 - **Vererbungsbeziehung (Inheritance):**
 - Bezeichnet das technische Übernehmen von Programmtext (Implementierung) von einer Oberklasse in eine Unterklasse.
 - Garantiert **keine Ersetzbarkeit** und sollte primär als Hilfsmittel, nicht als strukturierendes Element gesehen werden.
 - **Untertypbeziehung höherer Ordnung:** Wird für bestimmte Formen der Generizität benötigt (garantiert keine Ersetzbarkeit).
 - **Simulation:** 1. Das Abbilden eines Konzepts der realen Welt in Software. 2. Das Vergleichen der Ausdrucksstärke formaler Systeme.
-

Daten und Datenfluss

Arten von Variablen

- **Lokale Variable:**
 - Wird innerhalb einer Funktion/Methode deklariert.
 - Lebensdauer: Wird beim Aufruf angelegt und bei der Rückkehr freigegeben.
 - **Parameter:**
 - Ähnt einer lokalen Variable, wird aber vom Aufrufer mit einem **Argument** (Wert) initialisiert.
 - **Eingangsparameter:** Daten fließen nur hinein (Aufrufer $\rightarrow$ Aufgerufener).
 - **Durchgangsparameter:** Daten fließen in beide Richtungen (Aufrufer $\leftrightarrow$ Aufgerufener). Oft als Referenzparameter realisiert.
 - **Ausgangsparameter:** Daten fließen nur hinaus (Aufrufer $\leftarrow$ Aufgerufener).
 - **Variable in Modularisierungseinheit:**
 - z.B. **Objektvariable** (Instanzvariable) oder **Klassenvariable** (statische Variable).
 - Langlebig; Speicher wird meist durch Speicherbereinigung (Garbage Collection) freigegeben.
 - **Globale Variable:**
 - Langlebige Variable, die außerhalb von Modularisierungseinheiten existiert oder (im weiteren Sinn) eine Variable in einem statischen Modul (Klassenvariable).
-

Kommunikation über Variablen

- Kommunikation findet statt, wenn an einer Stelle ein Wert in eine Variable geschrieben wird und eine andere Stelle von dem gelesenen Wert abhängt.
 - **Gefahren:**
 - Gemeinsame Variablen (wie globale Variablen) sind mächtig, **unterlaufen** aber den klar definierten Kontrollfluss der strukturierten Programmierung.
 - Sie machen den Programmablauf **schwer durchschaubar** und stellen ein **Sicherheitsrisiko** dar.
 - **Referenzen (Zeiger) und Aliase:**
 - **Aliase** (wenn mehrere Variablen auf exakt dieselben Daten im Speicher zeigen) **verschleiern** den Programmablauf und die Kommunikation noch stärker.
 - **Lazy-Evaluation (Verzögerte Auswertung):**
 - Eine alternative Form der Programmsteuerung, bei der Kommunikation über Variablen die Kontrollstrukturen ersetzen kann.
 - Kontrollstrukturen bauen zunächst nur ein Netz von **Abhängigkeiten** zwischen Daten auf.
 - Die tatsächliche **Ausführung (Reduktion)** der Berechnung erfolgt erst dann, wenn das Ergebnis wirklich benötigt wird (z.B. bei einer Ausgabe oder in Iteratoren).
-

Strategien im Umgang mit der Kommunikation über Variablen

- **Prozedurale Programmierung:**
 - Ziel: Der Programmfluss soll dem (sichtbaren) Kontrollfluss entsprechen.
 - Globale Variablen und Aliase sind zwar erlaubt und oft nötig, gelten aber als **unerwünscht** und sollen minimiert werden.
 - Prozeduren oder Objekte werden **nicht als Daten** (z.B. als Parameter) verwendet.
- **Funktionale Programmierung:**
 - Ziel: **Funktionen sind Daten** und ersetzen die imperativen Kontrollstrukturen.
 - **Seiteneffekte (destruktive Zuweisungen) sind verboten.**
 - Dadurch stören Aliase nicht (Konzept der **referentiellen Transparenz**: Es ist egal, ob man das Original oder eine Kopie hat).
- **Objektorientierte Programmierung:**
 - Ziel: Kombination aus Prozeduren (Methoden) und **Objekten als Daten**.
 - Umgang mit den Gefahren:
 1. **Abstraktion:** Objekte und Variablen werden als **nominale abstrakte Datentypen** verstanden (man verlässt sich auf die Semantik/den Vertrag, nicht auf die Implementierung).
 2. **Dynamisches Binden:** Erzwingt ein abstraktes Verständnis, da der konkrete Kontrollfluss zur Compilezeit oft unbekannt ist.
 3. **Eingrenzung:** Kommunikation über Variablen wird **örtlich eingegrenzt** (z.B. auf `private` Variablen innerhalb einer Klasse).

4. **Offensiver Umgang mit Aliasen:** Strikte Unterscheidung zwischen **Identität** (Speicheradresse, `==`) und **Gleichheit** (Zustand, `.equals()`).
-

Verteilung der Daten (Paradigmen-abhängig)

- **Prozedural:** Oft globale Datenstrukturen; jeder Wert ist meist nur auf *eine* Weise zugreifbar.
 - **Funktional:** Tendenz zu großen (unveränderlichen) Strukturen; änderbare Daten referenzieren oft stabile Daten.
 - **Objektorientiert:** Verschiedene, zusammengehörige Daten werden zu einem Objekt zusammengefasst (gekapselt).
 - **Verteilte Programmierung:** Daten sind (oft redundant) über viele Rechner verteilt. Ziel: Daten und die Berechnung (die sie braucht) zusammenbringen.
 - **Parallele Programmierung:** Ziel: Kurze Gesamtlaufzeiten durch Einsatz vieler Prozessoren. Daten werden typischerweise in Bereiche aufgeteilt, die **unabhängig** voneinander bearbeitet werden können.
 - **Nebenläufige Programmierung (Concurrency):** Ziel: Effiziente Reaktionsfähigkeit auf (gleichzeitige) Ereignisse. Verschiedene Handlungsstränge (Threads) sollen möglichst auf *unterschiedliche* Daten zugreifen, um Synchronisationsaufwand zu minimieren.
 - **Echtzeitprogrammierung:** Ziel: Garantierte Antwortzeiten. Verwendet keine zufallsbasierten Strukturen (z.B. Hashtabellen), da deren Laufzeit nicht garantiert werden kann.
 - **Metaprogrammierung:** Programme selbst werden als Daten behandelt und können zur Laufzeit analysiert oder modifiziert werden.
-

Typisierung

Typprüfungen

- **Arten von Typprüfungen (nach Zeitpunkt):**
 - **Statische Typisierung:** Der Compiler kennt (fast) alle Typen zur Übersetzungszeit und garantiert die Typkonsistenz.
 - *Vorteil:* Hilft beim Programmverständnis; keine Typfehler zur Laufzeit. *Nachteil:* Eingeschränkte Flexibilität.
 - **Starke Typisierung (z.B. Java):** Der Compiler kennt Typen nur teilweise (z.B. den deklarierten Typ einer Variable, aber nicht unbedingt den genauen dynamischen Typ). Er garantiert aber dennoch die Typkonsistenz (z.B. durch Mechanismen wie dynamisches Binden).
 - **Dynamische Typisierung:** Der Compiler kennt die Typen nicht. Die Konsistenz wird erst zur Laufzeit bei der Ausführung geprüft.
 - *Vorteil:* Hohe Flexibilität. *Nachteil:* Typfehler erst zur Laufzeit.
- **Darstellung von Typen im Programm:**

- **Explizit spezifiziert:** Typinformation wird hingeschrieben (z.B. `int x = 10;`). Dient als zuverlässige Form der Dokumentation.
- **Inferiert (Typinferenz):** Typinformation wird weggelassen (z.B. `var x = 10;`). Der Compiler leitet den Typ eindeutig aus dem Kontext ab.
 - *Global inferiert:* (z.B. Haskell). Ermöglicht statische Typisierung fast ohne Typangaben, funktioniert aber meist nicht mit dynamischem Binden.
 - *Lokal inferiert:* (z.B. `var` in Java). Der Typ wird aus der (explizit typisierten) Umgebung abgeleitet.

Typen und Entscheidungsprozesse

- **Explizite Typinformation** (z.B. `String name`) **reflektiert eine Design-Entscheidung und hält sie fest.**
- Je **früher** im Entwicklungsprozess eine Entscheidung (ein Typ) festgelegt wird, desto stabiler und effizienter ist das System tendenziell.
- **Zeitpunkte von Entscheidungen:**
 - **Sprachdefinition:** `int` ist eine 32-bit-Zweierkomplementzahl.
 - **Erstellung Übersetzungseinheit (Code schreiben):** Variable `x` ist vom Typ `int`.
 - **Einbinden statischer Module (Generizität):** Typparameter T wird durch `Integer` ersetzt.
 - **Compilation:** (Meist nur semantisch unbedeutende Optimierungen).
 - **Initialisierungsphase (Laufzeit-Start):** Prüfen, ob eine Konfigurationsdatei ein `Integer` enthält.
 - **Programmausführung (Laufzeit):** Dynamische Typabfrage `if (y instanceof Integer) ...`.
- Wenn eine frühe Entscheidung (ein Typ) geändert werden muss, hilft der Compiler, alle betroffenen Stellen zu finden (da sie Typfehler erzeugen).

Nominale und strukturelle Typen

- **Nominaler Typ:** Ein Typ, der einen eindeutigen **Namen** hat (z.B. `class Person {...}`).
- **Struktureller Typ:** Ein Typ, der **anonym** ist und nur durch seine **Struktur** (Methoden, Variablen) definiert wird.

Nominale Typen (z.B. Java-Klassen)	Strukturelle Typen (z.B. Go, TypeScript)
Typen sind gleich bei gleichem Namen .	Typen sind gleich bei gleicher Struktur .
An nur einer Stelle definiert.	An mehreren (impliziten) Stellen definierbar.
Unterstützt nominale Abstraktion (Fokus auf Bedeutung/Absicht).	Unterstützt strukturelle Abstraktion (Fokus auf Form/Signatur).

Nominale Typen (z.B. Java-Klassen)	Strukturelle Typen (z.B. Go, TypeScript)
Hohe Abstraktionsgrade (über Kommentare/DbC) möglich.	Abstraktionsgrad bleibt niedrig.
Untertypbeziehungen müssen explizit deklariert werden (<code>extends</code>).	Untertypbeziehungen sind implizit (Compiler prüft Struktur).
Programmierer:in sichert semantische Ersetzbarkeit zu (über Zusicherungen).	Garantiert nur strukturelle Ersetzbarkeit.
Name ist stellvertretend für Eigenschaften.	Man darf keine zusätzlichen Eigenschaften annehmen.
Gut lesbar (Absicht ist klar).	Weniger gut lesbar (Absicht unklar).
Erfordert hohe Programmierdisziplin (bei Ersetzbarkeit).	Ermöglicht einfache Verwendung.

Nominaler/Struktureller Typ vs. Abstraktion

Man kann die Konzepte "mischen", um bestimmte Ziele zu erreichen:

- **Nominale Abstraktion durch strukturellen Typ simulieren:**
 - Man fügt dem strukturellen Typ eine "Marker"-Methode hinzu (z.B. `iBeLongToStack()`).
 - Dadurch wird die **Struktur** einzigartig, obwohl man eigentlich den **Namen** (die Absicht) meint.
- **Strukturelle Abstraktion durch nominalen Typ simulieren:**
 - Man verwendet einen nominalen Typ (z.B. ein Java-Interface), stellt aber durch Kommentare (oder den Kontext) klar, dass nur die Struktur (Signatur) relevant ist.
 - Beispiel: **Funktionale Interfaces** in Java. Der Name (`Function`) ist fast egal; wichtig ist nur, dass es eine Methode `apply(T t)` gibt, die zum Lambda passt.

Rekursive Datenstrukturen

- Typen für potenziell unendlich große Datenstrukturen (wie Listen oder Bäume) müssen **induktiv konstruiert** werden.
- **Beispiel (Liste von Integern):**
 1. **Basisfall (M_0):** $M_0 = \{end\}$ (z.B. die leere Liste).
 2. **Induktionsschritt (M_{i+1}):** $M_{i+1} = M_i \cup \{elem(n, x) | n \in Int; x \in M_i\}$ (Eine neue Liste *elem* besteht aus einer Zahl *n* und einer **bereits existierenden** Liste *x*).
 3. **Gesamtmenge (M):** $M = \bigcup_{i=0}^{\infty} M_i$ (Die Vereinigung aller so konstruierbaren Listen).
- **Kürzere Darstellung (z.B. Haskell-Syntax):** `data IntList = end | elem(Int, IntList)`
- **Fundiertheit:** Die Basismenge M_0 darf nicht leer sein. Es muss immer mindestens eine **rekursionsfreie Alternative** (den Basisfall, z.B. `end`) geben.

- **Ansatz in Java:** In Java ist der Basisfall M_0 implizit `null`. Jede Klasse (z.B. `Node`) entspricht der Konstruktionsvorschrift für M_{i+1} . Deshalb muss man in Java bei rekursiven Strukturen immer `null` als Basisfall behandeln.
-

Statisches Propagieren von Eigenschaften

- Betrachten wir:
 - `public static int plusZwei(int x) { return x+2; }`
 - `int y = plusZwei(3);`
- Die Typkonsistenz (geprüft vom Compiler) garantiert, dass die Typen zusammenpassen (von `3` zu `x`, vom Ergebnis $x + 2$ zu `y`).
- Dabei werden **statisch bekannte Eigenschaften** (wie der Typ) vom Argument zum Parameter und vom Ergebnis zur Variable **propagiert** (weiterverbreitet).
- Dies gilt nicht nur für Typinformation, sondern für **jede statisch verfügbare Information**.
- Beispiel: Wenn der Compiler weiß, dass ein Wert garantiert *ungleich null* ist, kann diese Information an den aufgerufenen Parameter propagiert werden, wodurch dort eventuell ein `null`-Check entfallen kann.
- Der Compiler muss die Information nur (statisch) erkennen können, das Propagieren selbst ist einfach.

vo3

Etablierte Denkmuster und Werkzeugkisten

Prozedurale Programmierung

Alles im Griff

- **Namensgebung:** Basiert auf **Prozeduren** (Methoden), die sich gegenseitig aufrufen.
- **Programmfortschritt:** Wird durch **destruktive Zuweisungen** (Seiteneffekte) erzielt, die den (meist globalen) Programmzustand ändern.
- **Struktur:** Nutzt heute vorwiegend die **strukturierte Programmierung** (Sequenz, Auswahl, Wiederholung) anstelle von `goto`.
- **Fokus:** Liegt auf der **Programmierung im Feinen** (Details des Ablaufs). Programmierung im Groben (Architektur) beschränkt sich meist auf Module zur getrennten Übersetzung.
- **Datenbehandlung:** Weder Objekte noch Prozeduren werden als Daten im engeren Sinn verwendet.

Typische Eigenschaften:

- **Einfachheit:** Überschaubare Menge an Sprachkonzepten, anfängerfreundlich.
- **Volle Kontrolle:** Der Programmablauf und die Datenstruktur (z.B. Arrays, Records) werden durch den Programmierer exakt kontrolliert.
- **Hardwarenähe:** Die gute Kontrollmöglichkeit macht sie ideal für hardwarenahe Programmierung.
- **Kontrollzwang:** Jedes Detail *muss* festgelegt werden, auch wenn es unwichtig ist. Dies führt bei mittlerer Komplexität schnell zu unübersichtlichen Programmen.
- **Abstraktion:** Erfolgt fast nur über Prozeduren (λ -Abstraktion), nominale Abstraktion spielt eine untergeordnete Rolle.
- **Formale Korrektheit:** Lässt sich gut mit formalen Korrektheitsbeweisen kombinieren.

Beispiel: Prozeduraler Stil in Java

- Nutzt `static` Methoden als Prozeduren.
- Nutzt Klassen als Module (Container für statische Methoden/Variablen).
- Nutzt `static` Variablen als globale Variablen.
- Nutzt Objekte von Klassen ohne Methoden als "Records" (reine Datencontainer).

[pp25v03, p.3](#)

Totgesagte leben länger

Prozedurale Programmierung ist nicht überholt, sondern wird in bestimmten Bereichen bewusst eingesetzt:

- **Hardwarenahe Programmierung:** Unerlässlich, wenn spezifische Hardware oder Speicheradressen angesprochen werden müssen. Sprachen wie C, aber auch Assembler oder Forth werden hier genutzt.

- **Echtzeitprogrammierung:** Nötig, wenn maximale Antwortzeiten (Deadlines) garantiert werden müssen.
 - **Weiche Echtzeitsysteme:** Gelegentliches Verpassen der Deadline ist unschön, aber nicht katastrophal.
 - **Harte Echtzeitsysteme:** (Sicherheitskritisch) Die Einhaltung der Deadline ist absolut notwendig. Erfordert stark eingeschränkte prozedurale Stile, oft mit formalen Beweisen.
 - **Scripting:** Kleine, oft dynamisch typisierte Programme (z.B. Shell-Skripte, Python), die bestehende Programme aufrufen und kombinieren. Oft für den Einmalgebrauch oder enge Kontexte gedacht, nicht für langfristige Wartung.
 - **Micro-Services:** Eine Softwarearchitektur, bei der eine komplexe Anwendung in viele kleine, unabhängige Prozesse (Services) zerlegt wird. Da jeder Service klein und überschaubar ist, wird er intern oft prozedural implementiert.
 - **Hobby- & Anwendungsprogrammierung:** Der Einstieg erfolgt fast immer prozedural. Viele Entwickler bleiben bei diesem Stil und bringen wertvolles **Domain-Wissen** (Fachwissen über den Anwendungsbereich) in Projekte ein.
-

Objektorientierte Programmierung (OOP)

OOP unterstützt Softwareentwicklungsprozesse, die auf **inkrementeller Verfeinerung** (schrittweiser Verbesserung) und **leichter Wartbarkeit** aufbauen.

Modularisierungseinheiten und Abstraktion

- **Objekt:** Die grundlegende Modularisierungseinheit zur Laufzeit.
- **Eigenschaften eines Objekts:**
 1. **Identität (Identity):** Das Objekt ist eindeutig (z.B. durch seine Speicheradresse).
 2. **Zustand (State):** Die aktuellen Werte seiner (privaten) Variablen.
 3. **Verhalten (Behavior):** Wie das Objekt auf Nachrichten (Methodenaufrufe) reagiert.
- **Gleichheit vs. Identität:**
 - **Identisch:** Zwei Referenzen zeigen auf dasselbe Objekt im Speicher (`==`).
 - **Gleich:** Zwei (potenziell verschiedene) Objekte haben denselben Zustand und dasselbe Verhalten (`.equals()`).
- **Schnittstelle (Interface) vs. Implementierung:**
 - **Implementierung:** Der konkrete Programmcode, der das Verhalten im Detail festlegt.
 - **Schnittstelle:** Eine **abstrakte** Beschreibung des Verhaltens, die Details offen lässt. Sie dient als Vertrag (meist als nominale Abstraktion).
- **Datenabstraktion:** Entsteht durch **Kapselung** (Bündeln von Daten und Methoden) + **Data-Hiding** (Verstecken der Implementierung/Daten hinter der Schnittstelle).
- **Klasse:**
 - Eine Schablone zur Erzeugung neuer Objekte (Instanzen).
 - Legt Implementierung und Schnittstellen für alle ihre Instanzen fest.
 - Definiert **Konstruktoren** zur Initialisierung neuer Objekte.

Polymorphismus und Ersetzbarkeit

- **Polymorphismus (Vielgestaltigkeit):** Eine Variable oder Methode kann mehrere Typen haben.
[📖: pp25v03, p.6](#)
 - **Universeller Polymorphismus (gleiche Struktur):**
 - **Generizität:** (Parametrischer Polymorphismus) Code funktioniert für verschiedene Typen (z.B. `List<T>`).
 - **Untertypbeziehungen:** (Inklusionspolymorphismus) Code funktioniert für einen Typ *und* alle seine Untertypen (Basis der OOP).
 - **Ad-hoc-Polymorphismus (verschiedene Struktur):**
 - **Überladen:** Mehrere Methoden mit gleichem Namen, aber unterschiedlichen Parametern.
 - **Typumwandlung (Coercion):** Implizite Umwandlung (z.B. `int` zu `double`).
- **OOP-Polymorphismus:** Bezieht sich primär auf **Untertypbeziehungen**.
- **Deklarierte vs. Dynamische Typen:**
 - **Deklarierte Typen:** Der Typ, mit dem eine Variable im Code deklariert wurde (z.B. `Animal`
`a = ...`).
 - **Dynamische Typen:** Der spezifischste Typ des Objekts, das zur Laufzeit *tatsächlich* in der Variablen liegt (z.B. `new Dog()`).
- **Dynamisches Binden:**
 - Die Entscheidung, *welche* Methodenimplementierung ausgeführt wird (z.B. `Dog.bellen()` oder `Cat.miauen()`), wird erst zur **Laufzeit** basierend auf dem **dynamischen Typ** des Empfängers getroffen.
 - Dies ist ein Eckpfeiler der OOP.
- **Vererbung (Inheritance):**
 - Ein Mechanismus, bei dem eine Unterklasse (Subclass) von einer Oberklasse (Superclass) ableitet.
 - Dient primär der **Codewiederverwendung** durch **Erweiterung** (neue Methoden/Variablen) und **Überschreiben** (Ersetzen von Methoden).
 - In Java ist Vererbung eng mit Untertypbeziehungen verknüpft, aber **Vererbung ist nicht dasselbe wie Ersetzbarkeit!**

Typische Vorgehensweisen

- **Faktorisierung:** Das Zusammenfassen von zusammengehörenden Programmteilen (Variablen und Methoden) zu Modularisierungseinheiten (Objekten/Klassen).
 - **Ziel:** Gute Faktorisierung, sodass Programmänderungen möglichst nur an einer Stelle (lokal) nötig sind.
 - **Simulation:** Eine gute Faktorisierung simuliert oft Objekte der realen Welt.
- **Entwicklungsprozesse:** OOP eignet sich gut für **zyklische Prozesse** (statt Wasserfallmodell), da frühes Feedback und schrittweise Verfeinerung (Inkrementelles Vorgehen) unterstützt werden.
- **Erfahrung:** Der gezielte Einsatz von Erfahrung ist in der OOP essenziell, um aus den vielen Lösungsmöglichkeiten die zukunftsfähigste auszuwählen.

- **Faustregeln für gute Faktorisierung:**
 - **Hoher Klassen-Zusammenhalt (High Cohesion):** Die Verantwortlichkeiten (Methoden/Variablen) *innerhalb* einer Klasse sollen eng zusammenarbeiten und einem klaren Zweck dienen.
 - **Schwache Objekt-Kopplung (Low Coupling):** Die Abhängigkeiten *zwischen* verschiedenen Objekten sollen minimal sein.
- **Refaktorisierung (Refactoring):** Das Ändern der *Struktur* eines Programms, **ohne dessen Funktionalität (Verhalten) zu ändern**. Dient der Verbesserung der Faktorisierung (Erhöhung Kohäsion, Senkung Kopplung).

Worum es geht

- **Kernidee:** OOP eignet sich für die Entwicklung **langlebiger, komplexer Systeme** durch Profis. Der Schlüssel ist **Abstraktion auf hohem Niveau**.
- **Abstraktion in OOP:**
 - Objekte werden als **nominale abstrakte Datentypen** betrachtet (ihre Bedeutung/Semantik ist wichtiger als ihre Implementierung).
 - Wir arbeiten in einer "Phantasiewelt" von Software-Abbildern realer Dinge.
- **Herausforderungen:**
 - Diese "Phantasiewelt" muss realen Zwängen genügen (z.B. dem **Ersetzbarkeitsprinzip**).
 - Der Compiler kann nicht prüfen, ob unsere nominalen Abstraktionen (unsere "Ideen") logisch konsistent sind; **wir tragen die volle Verantwortung**.
 - Ein Verstoß an einer Stelle (z.B. Verletzung der Ersetzbarkeit) kann das gesamte System ("Kartenhaus") zum Einsturz bringen.
- **Evolution des Paradigmas:**
 - **Früher (bis 1990er):** Intuitiv, "Is-a" wurde fälschlich mit Ersetzbarkeit gleichgesetzt, oft dynamisch typisiert.
 - **Heute (Professionell):** Fokussiert auf das strikte Ersetzbarkeitsprinzip, bevorzugt stark typisierte Sprachen, Vererbung wird von Untertypbeziehungen klar unterschieden.

Funktionale Programmierung (FP)

Sauberkeit aus Prinzip (Purity)

- **Grundidee:** FP basiert auf Funktionen als Daten und eliminiert "unsaubere" Praktiken der prozeduralen Programmierung.
- **Vermeidung von Seiteneffekten:**
 - Der Haupt-Störfaktor ist die **destruktive Zuweisung** (Änderung eines Variablenwerts).
 - In reiner FP wird darauf verzichtet. Eine Variable ist nur ein Name für einen unveränderlichen Wert.
 - **Statt Schleifen** (die destruktive Zähler-Updates benötigen) wird **Rekursion** verwendet (jeder Aufruf erzeugt neue Parameter/Variablen).
- **Referentielle Transparenz:**

- Eine Konsequenz des Verzichts auf Seiteneffekte.
- Es gibt **keinen Unterschied mehr zwischen Gleichheit** (`.equals()`) **und Identität** (`==`). Ein Wert ist ein Wert, egal ob er kopiert wurde oder das Original ist.
- Dies macht Aliase (Referenzen auf dieselben Daten) harmlos.
- **Vorteile:**
 - Der Programmablauf wird nur durch den Kontrollfluss (Funktionsaufrufe) bestimmt, nicht durch versteckte Kommunikation über globale Variablen.
 - Gleiche Werte bleiben immer gleich.
 - Ermöglicht **Typinferenz** (automatisches Erkennen von Typen durch den Compiler ohne explizite Deklarationen).
 - Unterstützt **Pattern-Matching** (komplexe Fallunterscheidungen basierend auf der *Struktur* von Daten, nicht nur auf Werten).

Beispiel: Einfacher funktionaler Stil in Java

- Verwendet `final` für Variablen, um Unveränderlichkeit anzudeuten.
- Nutzt Rekursion statt Schleifen.
- Datenstrukturen (wie `Points` und `Stud`) sind unveränderlich (immutable); Änderungen erzeugen neue Objekte statt alte zu modifizieren.
- Funktionen werden als Daten simuliert, indem Objekte von Interfaces (wie `Func`) übergeben werden, die nur eine Methode enthalten.

[pp25v03, p.14](#)

[pp25v03, p.15](#)

Streben nach höherer Ordnung (Applikative Programmierung)

- **Applikative Programmierung:** Ein Programmierstil, bei dem **Funktionen höherer Ordnung** (Funktionen, die Funktionen als Parameter nehmen oder zurückgeben) als Ersatz für Kontrollstrukturen (wie Rekursion oder Schleifen) verwendet werden.
- **Ziel:** Statt Algorithmen neu zu schreiben, werden bestehende, vorgefertigte Funktionen (z.B. `map`, `filter`, `reduce`) geschickt **angewendet** und **kombiniert**.
- **Java 8 Streams & Lambdas:**
 - Java unterstützt diesen Stil durch **Java 8 Streams** (eine Sammlung von Funktionen höherer Ordnung für Datenmengen).
 - **Lambda-Ausdrücke** (z.B. `s -> s.length()`) sind eine kompakte Syntax, um anonyme Funktionen (Objekte von funktionalen Interfaces) zu erzeugen.
- **Ablauf (Map-Reduce):**
 1. **Strom-Erzeugung:** Eine Datenquelle wird in einen Strom umgewandelt (z.B. `list.stream()`).
 2. **Strom-Modifizierung (Map):** Beliebige Operationen filtern (`filter`) oder transformieren (`map`) die Daten im Strom (z.B. `stream.map(Integer::parseInt)`).
 3. **Strom-Abschluss (Reduce):** Eine Operation sammelt (`collect`) oder reduziert (`reduce`) die Elemente zu einem Endergebnis (z.B. `.reduce(0, (i, j) -> i + j)`).

- **Lazy Evaluation:** Modifizierende Operationen werden nicht sofort ausgeführt, sondern erst, wenn die abschließende Operation das Ergebnis anfordert.

Beispiel: Applikativer Stil in Java (Java 8 Streams)

- Nutzt `stream()`, `map()`, `collect()` etc. statt expliziter Rekursion oder Schleifen.
 - Nutzt Lambda-Ausdrücke (z.B. `s -> line(s, lm)`) und Methodenreferenzen (z.B. `HashMap::new`) für Operationen.
- [pp25v03, p.18](#)

Friedliche Koexistenz

- FP konkurriert eher mit der prozeduralen Programmierung (Programmierung im Feinen) als mit der OOP (Programmierung im Groben/Architektur).
- FP eignet sich gut für algorithmisch komplexe Aufgaben und Parallelisierung, aber weniger für hardwarenahe Steuerung.
- **Kombination mit OOP:** In modernen Sprachen wie Java koexistieren die Paradigmen.
 - Die Architektur ist objektorientiert.
 - Innerhalb von Methoden werden aber prozedurale (imperative) oder funktionale (applikative) Stile verwendet.
- **Herausforderung bei der Kombination:** Widersprüche müssen gehandhabt werden (z.B. Seiteneffekte vs. Referentielle Transparenz).
- **Lösung:** Strikte Trennung. OO-Methoden (mit Seiteneffekten) dürfen reine FP-Funktionen aufrufen, aber reine FP-Funktionen dürfen keine Methoden mit Seiteneffekten aufrufen.
- **Java-Streams & Seiteneffekte:** Java-Streams sind nicht *rein* funktional.
 - Abschließende Operationen (z.B. `collect`, `forEach`) haben oft Seiteneffekte (z.B. Befüllen einer Liste).
 - Modifizierende Operationen (`map`, `filter`) sollten seiteneffektfrei sein, da ihr Ausführungszeitpunkt (wegen Lazy-Evaluation) und ihre Ausführungsreihenfolge (bei parallelen Streams) unvorhersehbar sind.

Parallele Programmierung

Charakter der parallelen Programmierung

- **Ziel:** Möglichst **kurze Rechenzeit** (Gesamtlaufzeit) durch die Ausnutzung paralleler Hardware (mehrere Prozessoren/Kerne).
- **Werkzeuge:** Es gibt eine breite Palette an Hardware (z.B. Multi-Core, GPUs, Cluster) und Software-Werkzeugen, um dieses Ziel zu erreichen.
- **Messung (Speedup):** Der Erfolg wird durch den Speedup S_p gemessen:
 - $$S_p = T_1 / T_p$$
 - T_1 ist die Zeit auf **einer** Recheneinheit (sequenziell).
 - T_p ist die Zeit auf **p** Recheneinheiten (parallel).
- **Grenzen des Speedup (Amdahls Gesetz):**
 - Parallelisierung erzeugt immer **Zusatzaufwand** (Verwaltung, Synchronisation, Kommunikation). Die **Summe** der Rechenzeit aller p Einheiten ist daher höher als T_1 .
 - Daraus folgt: $S_p < p$. Ein Speedup von > 1 wird nur erreicht, wenn die Parallelisierung diesen Zusatzaufwand überkompensiert.
- **Einsatzgebiet:** Lohnt sich meist nur für die Verarbeitung **großer Datenmengen** mit **einheitlicher Struktur**.
- **Kernaufgabe:** Die wichtigste Aufgabe ist es, eine Vorgehensweise zu finden, die die Daten in möglichst **unabhängige Blöcke** zerlegt. Für häufige Probleme (z.B. lineare Algebra) gibt es dafür hochoptimierte, fertige Bibliotheken.

Naive Parallelisierung und ihre Tücken

Das naive Ersetzen von `stream()` durch `parallelStream()` in Java kann die Ausführung oft **verlangsamen** statt beschleunigen.

- **Ursachen für längere Laufzeit (Zusatzaufwand):**
 - **Hoher Verwaltungsaufwand:** Das Aufspalten der Daten auf Threads, das Umschalten zwischen Aufgaben (Context Switching) und das spätere Zusammenfassen der Ergebnisse kostet Zeit. Die einzelnen Aufgaben dürfen nicht zu klein sein.
 - **Gemeinsamer Speicher:** Wenn Threads auf dieselben Daten zugreifen, müssen diese Zugriffe koordiniert werden, um **Race-Conditions** (Ergebnis hängt von der unvorhersehbaren Reihenfolge der Threads ab) zu vermeiden.
 - **Datenabhängigkeiten:** Wenn Aufgabe B das Ergebnis von Aufgabe A benötigt, muss B warten. Dies erfordert **Synchronisation**, was die Parallelität aufhebt (die Threads laufen **hintereinander** statt nebeneinander).
 - **Liveness-Probleme:** Extreme Verzögerungen oder Stillstand durch:
 - **Starvation (Verhungern):** Eine wichtige Aufgabe erhält keine Rechenzeit.

- **Deadlock (Verklemmung):** Threads blockieren sich gegenseitig, weil sie im Kreis aufeinander warten (z.B. A wartet auf B, B wartet auf A).
 - **Livelock:** Threads arbeiten aktiv, erzielen aber keinen Fortschritt (z.B. weichen sich ständig gegenseitig aus).
 - **Scheduling:** Die Zuteilung der Aufgaben (Tasks) zu den Prozessorkernen (Scheduling-Strategie) ist komplex und nicht immer optimal kontrollierbar.
 - **Zerlegung (Amdahls Gesetz):** Jedes Programm hat **sequentielle Teile** (z.B. das Einlesen der Daten oder das Zusammenfassen des Endergebnisses), die nicht parallelisiert werden können. Diese bestimmen die minimal erreichbare Laufzeit, egal wie viele Kerne man einsetzt.
 - **Engpässe (Bottlenecks):** Oft wird das System ungleichmäßig ausgelastet. Wenn alle Threads z.B. gleichzeitig auf die langsame Festplatte oder das Netzwerk warten, bringt die Parallelisierung der CPU-Berechnung nichts.
-

Formen der Parallelität

Hardware

- **Prozessorkern (Implizite Parallelität):** Moderne Kerne führen *innerhalb eines* Befehlsstroms mehrere Befehle gleichzeitig aus (z.B. VLIW, Superskalar-Architektur).
- **MIMD (Multiple Instruction Multiple Data):** Mehrere Prozessoren/Kerne führen *unabhängige* Befehlsströme aus.
 - **Multi-Core-Prozessor:** Mehrere Kerne auf einem Chip. Teilen sich oft die Speicheranbindung, was zum Engpass werden kann.
 - **UMA (Uniform Memory Access) / SMP:** Mehrere Prozessoren greifen über eine (aufwändige) Hardware *gleich schnell* auf einen gemeinsamen Speicher zu.
 - **NUMA (Non-Uniform Memory Access) / DSM:** Jeder Prozessor hat *eigenen, schnellen* Speicher, kann aber auch *langsamer* auf den Speicher anderer Prozessoren zugreifen.
 - **Rechnernetzwerk:** Kommunikation erfolgt über ein (langsames) Netzwerk. Wird meist für *verteilte Programmierung* (hohe Last bewältigen) statt *paralleler Programmierung* (eine Aufgabe schnell lösen) genutzt.
- **SIMD (Single Instruction Multiple Data):** *Ein* Befehl wird auf *viele* Daten gleichzeitig angewendet (z.B. Vektoreinheiten in CPUs, GPUs).

Software

- **Prozess:** Eine Ausführungseinheit des **Betriebssystems**. Prozesse sind durch die Hardware (MMU - Memory Management Unit) **stark voneinander abgeschirmt** und haben ihren eigenen virtuellen Speicher.
- **Thread:** Eine Ausführungseinheit **innerhalb eines Prozesses**. Threads sind leichtgewichtiger, teilen sich aber den **gemeinsamen Speicher** des Prozesses und sind *nicht* voneinander abgeschirmt (erfordern daher explizite Synchronisation).

Interprozesskommunikation (IPC)

Methoden für Prozesse, um Daten auszutauschen:

- **Dateien:** Prozesse schreiben und lesen aus gemeinsamen Dateien. Synchronisation erfolgt z.B. über Locks oder das (langsame) Öffnen/Schließen.
 - **Pipelines, Sockets:** Datenströme (ähneln Dateien, potenziell unendlich). Die Synchronisation geschieht implizit durch die Datenverfügbarkeit (Lesen blockiert, bis Daten geschrieben wurden).
 - **Shared-Memory (Gemeinsamer Speicher):** Mehrere Prozesse erhalten Zugriff auf denselben physischen Speicherbereich. Sehr schnell, aber systemabhängig und erfordert explizite Synchronisation (z.B. Semaphore).
-

Beispiel: Primzahlberechnung (Parallel)

- **Algorithmus:** Sieb des Eratosthenes (findet Primzahlen durch Aussieben von Vielfachen).
 - **Parallelisierungsidee:** Die Daten (Zahlen) werden in unabhängige Bereiche aufgeteilt, die parallel verarbeitet werden können.
 - **Implementierung (Par.java):**
 - `main` berechnet Primzahlen in Phasen (z.B. erst bis 256, dann bis 256^2 , etc.).
 - `inRange(low, high)` nutzt einen **parallelen Stream** (`(LongStream.range(...).parallel())`) um Zahlen in einem Bereich zu prüfen.
 - `isPr(n)` (der Filter) prüft eine einzelne Zahl n sequenziell gegen die *bereits bekannten* Primzahlen (aus `prims`).
[pp25v04, p.5](#)
 - **Kernaufgabe:** Die Schwierigkeit liegt nicht im `parallel()`-Aufruf, sondern in der **Zerlegung der Aufgabe** (hier in Phasen), sodass die Daten (`prims`) während der parallelen `filter`-Operation nicht geändert werden müssen (Vermeidung von Abhängigkeiten).
-

Nebenläufige Programmierung (Concurrency)

- **Charakter:** Viele gleichzeitig oder überlappt ablaufende **Handlungsstränge** (oft *Threads* genannt).
 - **Ziel:** Nicht primär Speedup, sondern **Vereinfachung des Programms** (z.B. ein Thread pro Client in einem Webserver) oder **Reaktionsfähigkeit** auf unvorhersehbare Ereignisse.
 - **Ausführung:** Handlungsstränge werden häufig durch **Warten** (auf E/A, auf Ereignisse, auf Locks) unterbrochen.
 - **Ressourcen:** Die Anzahl der Threads richtet sich nach dem **Bedarf** der Anwendungslogik, nicht (nur) nach der Anzahl der Prozessorkerne.
 - **Herausforderung:** Zugriffe auf gemeinsame Daten sind oft unvermeidbar und erfordern **Synchronisation**, um Race-Conditions zu verhindern. Die Vermeidung von **Liveness-Problemen** (Deadlock etc.) ist schwierig.
-

Beispiel: Simulation Bakterienwachstum (BactSim)

- Jedes Bakterium ist ein eigener `Thread` (implementiert `Runnable`).
 - Threads werden dynamisch erzeugt (Vermehrung) und beendet (Sterben).
 - `Thread.sleep(...)` simuliert Warten (z.B. auf Wachstum) und gibt Rechenzeit für andere Threads frei.
 - **Synchronisation:** Der Zugriff auf einen "Platz" in der Petrischale (`field[x][y]`) muss synchronisiert werden, da mehrere Bakterien (Threads) gleichzeitig darauf zugreifen könnten.
 - `synchronized` Methoden (`occupy`, `gone`) garantieren **gegenseitigen Ausschluss (Mutual Exclusion)**. Nur ein Thread kann *gleichzeitig* eine `synchronized`-Methode *desselben* State-Objekts ausführen.
-

Das Monitor-Konzept in Java

Java nutzt *Monitore* (implementiert über Locks an jedem Objekt) zur Synchronisation.

- `synchronized(obj) {...}` (**Synchronized-Block**):
 - Der Block ist ein **kritischer Abschnitt**.
 - Ein Thread "lockt" das Objekt `obj` beim Betreten.
 - Andere Threads, die dasselbe `obj` locken wollen, **warten** (blockieren), bis der Lock freigegeben wird.
 - **Rekursion:** Ein Thread, der den Lock bereits hält, kann ihn problemlos erneut anfordern (z.B. bei rekursiven Aufrufen).
 - `synchronized R m(...){...}` (**Synchronized-Methode**):
 - Abkürzung für einen `synchronized(this)`-Block um den gesamten Methodenrumpf.
 - **Warten auf Bedingung (Koordination):**
 - `wait()`: Muss *innerhalb* eines `synchronized`-Blocks aufgerufen werden. Der Thread gibt den Lock frei und geht in einen Wartezustand.
 - `notifyAll()`: Muss *innerhalb* eines `synchronized`-Blocks aufgerufen werden. Weckt *alle* Threads auf, die auf diesem Objekt via `wait()` warten.
 - `notify()`: Weckt nur *einen* wartenden Thread auf (riskant, da es der "falsche" sein könnte).
 - **Wichtig:** `wait()` muss **immer in einer** `while`-**Schleife** aufgerufen werden (`while (!bedingung) wait();`), da ein Thread jederzeit (auch "grundlos") aufwachen kann (Spurious Wakeup) oder die Bedingung durch einen anderen Thread bereits wieder geändert wurde.
-

Beispiel: Producer-Consumer (Buffer)

- **Problem:** Ein oder mehrere *Producer*-Threads erzeugen Daten, ein oder mehrere *Consumer*-Threads verbrauchen sie. Ein Puffer (Buffer) synchronisiert den Zugriff.

- `Buffer.put(v)`:
 - Muss warten, wenn der Puffer voll ist (`buffer[in] != null`).
 - Verwendet `wait()`, um zu blockieren.
 - Trägt den Wert `v` ein und ruft `notifyAll()` auf, um wartende Consumer zu wecken.
- `Buffer.get()`:
 - Muss warten, wenn der Puffer leer ist (`buffer[out] == null`).
 - Verwendet `wait()`, um zu blockieren.
 - Entnimmt einen Wert und ruft `notifyAll()` auf, um wartende Producer zu wecken.

pp25v04, p.9

pp25v04, p.10

Liveness-Probleme

- **Starvation (Verhungern):**
 - **Problem:** Ein wichtiger Programmteil/Thread bekommt nicht genug Ressourcen (z.B. Rechenzeit), weil unwichtige Teile alles blockieren.
 - **Auswirkung:** Langsamer Fortschritt oder Stillstand.
 - **Lösung:** Bessere Ressourcenverteilung, z.B. Puffer.
- **Deadlock (Verklemmung):**
 - **Problem:** Mehrere Threads warten zyklisch aufeinander (z.B. A hält Lock 1 und wartet auf Lock 2; B hält Lock 2 und wartet auf Lock 1).
 - **Auswirkung:** Dauerhafte Blockade, kein Fortschritt.
 - **Lösung:** Timeouts; oder: Locks immer in einer global festgelegten Reihenfolge anfordern.
- **Livelock:**
 - **Problem:** Ähnlich wie Deadlock, aber Threads warten nicht passiv, sondern reagieren aktiv auf den Zustand des anderen (z.B. zwei Leute weichen sich in einem Gang immer in dieselbe Richtung aus).
 - **Auswirkung:** Kein Fortschritt trotz Aktivität.
 - **Lösung:** Timeouts, kein "aktives Warten" (Spinning).

Strukturelle Untertypbeziehungen

Ersetzbarkeitsprinzip

- **Definition:** Ein Typ U ist ein Untertyp eines Typs T , wenn eine Instanz von U **überall dort verwendbar** ist, wo eine Instanz von T erwartet wird.
- **Wo wird es benötigt?**
 1. Bei **Zuweisungen** ($y = x$): Der (dynamische) Typ von x muss ein Untertyp des (deklarierten) Typs von y sein.

2. Bei **Methodenaufrufen**: Der (dynamische) Typ eines Arguments muss ein Untertyp des (deklarierten) Typs des formalen Parameters sein.
-

Regeln für strukturelle Untertypbeziehungen

Damit U ein Untertyp von T ist (basierend rein auf der Struktur/Signatur):

1. Konstanten/Variablen:

- **Konstante** (nur lesbar) in T (Typ A) $\rightarrow$ Konstante in U (Typ B): B muss Untertyp von A sein.
- **Variable** (les-/schreibbar) in T (Typ A) $\rightarrow$ Variable in U (Typ B): A und B müssen **äquivalent** sein.

2. Methoden:

- Für jede Methode in T muss es eine entsprechende Methode in U geben mit:
 - Gleicher Parameteranzahl und -arten (Eingang, Ausgang, ...).
 - **Ergebnistyp**: Ergebnistyp in U muss Untertyp von Ergebnistyp in T sein.
 - **Ausnahmen**: Methode in U darf **nicht mehr** Ausnahmen auslösen als die in T .
 - **Parametertypen**: Müssen zueinander passen (siehe unten).
-

Untertypen nach Parameterarten

Wenn eine Methode in T einen Parameter vom Typ A hat und die überschreibende Methode in U an derselben Stelle einen Parameter vom Typ B :

- **Eingangsparameter**: (Datenfluss: Aufrufer $\rightarrow$ Methode)
 - A muss Untertyp von B sein.
 - **Begründung**: Die Methode in U muss alles akzeptieren, was die Methode in T akzeptiert. Wenn der Aufrufer (der T erwartet) ein A übergibt, muss U (das B erwartet) dieses A annehmen können. Das geht nur, wenn A ein Untertyp von B ist.
 - **Ausgangsparameter**: (Datenfluss: Methode $\rightarrow$ Aufrufer)
 - B muss Untertyp von A sein.
 - **Begründung**: Der Aufrufer (der T erwartet) muss das Ergebnis von U (Typ B) als A behandeln können. Das geht nur, wenn B ein Untertyp von A ist.
 - **Durchgangsparameter**: (Datenfluss in beide Richtungen)
 - Muss gleichzeitig Ein- und Ausgangsparameter sein.
 - A und B müssen **äquivalent** sein.
-

Varianz von Typen

- **Kovarianz**: "Variiert gleichsinnig". A verhält sich zu B wie T zu U (d.h. B ist Untertyp von A).
 - Gilt für: **Ergebnistypen**, Ausgangsparameter, Konstanten (nur lesbare Elemente).

- **Kontravarianz:** "Variiert gegensinnig". A verhält sich zu B umgekehrt wie T zu U (d.h. A ist Untertyp von B).
 - Gilt für: **Eingangsparameter** (nur schreibbare Elemente).
- **Invarianz:** Gleichzeitig ko- und kontravariant. A und B müssen äquivalent sein.
 - Gilt für: **Variablen** (les- und schreibbare Elemente), Durchgangsparameter.

: pp25v04, p.16

- **Anmerkung:** Das Beispiel `class U extends T { public U meth(Tp) ... }` (wobei Tp Obertyp von Up ist) zeigt korrekte Varianz (kovarianter Rückgabety U , kontravarianter Parametertyp Tp). **In Java ist dies jedoch nicht erlaubt**; Java erlaubt nur kovariante Rückgabety, aber Parametertypen müssen invariant (exakt gleich) sein. Eine solche Signatur würde in Java als **Überladen**, nicht als **Überschreiben** behandelt.

Untertypbeziehungen in der Praxis

Theorie vs. Praxis

- Die **Theorie** der strukturellen Typen ist klar und widerspruchsfrei.
- Die **Praxis** (OOP) benötigt **nominale Typen** (Abstraktion über Namen/Bedeutung).
- Daher wird in Java (und ähnlichen Sprachen) eine **explizite Vererbungsbeziehung** (`extends` / `implements`) vorausgesetzt.
 - Dies verhindert "zufällige" Untertypbeziehungen (wenn zwei Klassen strukturell gleich, aber semantisch verschieden sind).
- **Einschränkung in Java:** Java erzwingt (fast) **Invarianz**.
 - Nur **Ergebnistypen** dürfen **kovariant** sein.
 - Alle anderen Typen (insb. Parameter) müssen **invariant** (exakt gleich) sein.
 - **Grund:** Dies ist intuitiver und vermeidet Konflikte mit dem Überladen von Methoden.

Grenzen von Untertypbeziehungen: Kovariante Probleme

- **Problem:** Oft wünscht man sich in der Praxis **kovariante** Eingangsparametertypen, obwohl die Theorie **Kontravarianz** vorschreibt.
- **Beispiel** `Point2D` / `Point3D`:

: pp25v04, p.21

 - Man möchte, dass `Point3D` von `Point2D` erbt.
 - Man möchte `equal` so überschreiben: `public boolean equal(Point3D p)`.
 - Dies ist **kovariant** zum Parameter `Point2D p` in der Oberklasse.
 - **Das ist FALSCH** und verletzt die Ersetzbarkeit.
 - In Java wird diese Methode die `equal(Point2D p)`-Methode nicht **überschreiben**, sondern nur **überladen**.

Code-Wiederverwendung durch Ersetzbarkeit

- **Zwischen Generationen:**

- Wenn Typen (Schnittstellen) stabil bleiben, können alte Clients neue Implementierungen (Untertypen) nutzen, ohne geändert zu werden.
- Beispiel: Eine Anwendung, die `Treiber 1` nutzt, kann ohne Neukompilierung `Treiber 2a` oder `Treiber 3a` verwenden, wenn diese Untertypen sind.

[📖: pp25v04, p.22](#)

- **Innerhalb eines Programms:**

- Code, der gegen einen Obertyp (z.B. `Person`) geschrieben ist, funktioniert automatisch für alle Untertypen (z.B. `Student`, `Tutor`, `Werkstudent`).
- Beispiel: Eine Methode `sendeSerienbrief(Person p)` funktioniert für alle.

[📖: pp25v04, p.23](#)

- **Faustregel:** Typen (Schnittstellen) sollten **stabil** sein, besonders weit oben in der Hierarchie.

Dynamisches Binden

- Der Aufruf `x.foo()` führt die Implementierung von `foo` aus, die zum **dynamischen Typ** von `x` gehört (nicht zum deklarierten).

- **Beispiel (Abstrakt):**

[pp25v04, p.24](#)

- `test(new A())` ruft `A.foo1()` und `A.foo2()` (welches `A.fooX()` ruft) auf.
Ausgabe: `foo1A`, `foo2A`.
- `test(new B())` (wobei `x` den deklarierten Typ `A` hat):
 - `x.foo1()`: Dynamisches Binden ruft `B.foo1()`. Ausgabe: `foo1B`.
 - `x.foo2()`: Dynamisches Binden ruft `A.foo2()` (da nicht in B überschrieben).
 - `A.foo2()` ruft `fooX()`.
 - **Wichtig:** Der Aufruf von `fooX()` erfolgt **wiederum dynamisch** auf dem Objekt `x` (das ja vom Typ `B` ist) und ruft daher `B.fooX()`. Ausgabe: `foo2B`.

- **Dynamisches Binden statt `switch`:**

- Statt einer `switch`-Anweisung über eine "Art"-Variable (`int anredeArt`)...
[pp25v04, p.25](#)
- ...verwendet man besser eine Typhierarchie (`Adressat`, `WeiblicherAdressat`, `MaennlicherAdressat`) und überschreibt eine Methode (`gibAnredeAus`).
[pp25v04, p.26](#)
- **Vorteil:** Besser lesbar (Namen statt Zahlen) und **erweiterbar**. Wenn eine neue Anredeform (z.B. `Firma`) hinzukommt, muss nur eine neue Klasse hinzugefügt werden; der alte Code, der `gibAnredeAus()` aufruft, muss nicht geändert werden. Bei `switch` müssten **alle** `switch`-Anweisungen im gesamten Code gefunden und angepasst werden.

Zusicherungen (Assertions)

Client-Server-Beziehungen

- Ein **Server** bietet Dienste an (z.B. die Ausführung einer Methode).
- Ein **Client** nutzt diese Dienste.
- Ein Objekt kann gleichzeitig Client und Server sein.

Vertrag (Contract)

- Zwischen Client und Server besteht ein "Vertrag" (Stichwort: **Design-by-Contract**).
- **Der Client erfüllt:** Vorbedingungen (Preconditions).
- **Der Server erfüllt:** Nachbedingungen (Postconditions) und Invarianten (Invariants).
- **Beide erfüllen:** History-Constraints.

Arten von Zusicherungen

1. Vorbedingung (Precondition)

- **Verantwortlich:** Client
- **Wann:** *Vor* dem Methodenaufruf.
- **Was:** Hauptsächlich Eigenschaften von Argumenten (z.B. "Array ist aufsteigend sortiert") oder manchmal der (sichtbare) Zustand des Servers (z.B. "Konto ist nicht überzogen").

2. Nachbedingung (Postcondition)

- **Verantwortlich:** Server
- **Wann:** *Vor* der Rückkehr aus dem Methodenaufruf.
- **Was:** Eigenschaften von Methodenergebnissen sowie Änderungen und Eigenschaften des Objektzustands.
- Klingt oft wie eine Methodenbeschreibung (z.B. "Methode fügt Element ein. Ergebnis ist 'true', falls Element bereits vorher in Menge war").

3. Invariante (Invariant)

- **Verantwortlich:** Server
- **Wann:** *Vor und nach* der Ausführung von Methoden (also in jedem stabilen Zustand).
- **Was:** Unveränderliche Eigenschaften von Objekten und Variablen (z.B. "Guthaben am Spargbuch ist immer eine positive Zahl").
- Die Gültigkeit kann von Bedingungen abhängen (z.B. "`zuverlaessig == false`" wenn Konto überzogen").
- Eine Invariante *impliziert* die Nachbedingung (da der Zustand nach der Methode die Invariante erfüllen muss).
- **Ausnahme:** Wenn eine Variable von außen (durch den Client) schreibbar ist, sind *Client und Server* für die Einhaltung der Invariante dieser Variable verantwortlich.

4. History-Constraint

- **Server-kontrollierter History-Constraint:**
 - **Verantwortlich:** Server
 - **Wann:** *Nach* der Methodenausführung (im Bezug auf den Zustand *vor* der Ausführung).
 - **Was:** Einschränkungen auf *Veränderungen* von Variablen (z.B. "Zählerwert erhöht sich mit jedem Methodenaufruf").
 - Impliziert die Nachbedingung.
- **Client-kontrollierter History-Constraint:**
 - **Verantwortlich:** Client
 - **Wann:** *Vor* den Methodenaufrufen.
 - **Was:** Einschränkungen auf die *Reihenfolge* von Aufrufen (z.B. "zuerst `initialize` aufrufen, dann erst andere Methoden" oder "nach jedem Aufruf von `a` ist ein Aufruf von `b` nötig").
 - Dies ist oft mit Synchronisation (Koordination) verbunden.

: pp25v05, p.8

: pp25v05, p.9

Zusicherungen und Ersetzbarkeit (Liskovsches Substitutionsprinzip)

Damit ein Typ U ein Untertyp von T sein kann (also U T ersetzen kann), müssen die Zusicherungen folgende Regeln einhalten:

1. **Vorbedingungen:** Dürfen im Untertyp U *schwächer oder gleich* sein wie in T .
 - (Der Untertyp U verlangt *weniger* oder das Gleiche vom Client als T).
 - Bei Vererbung: Verknüpfung mit `oder`.
2. **Nachbedingungen:** Müssen im Untertyp U *stärker oder gleich* sein wie in T .
 - (Der Untertyp U *garantiert mehr* oder das Gleiche wie T).
 - Bei Vererbung: Verknüpfung mit `und`.
3. **Invarianten:** Müssen im Untertyp U *stärker oder gleich* sein wie in T .
 - (Ausnahme: Bei von außen schreibbaren Variablen müssen sie *gleich* sein).
 - Bei Vererbung: Verknüpfung mit `und`.

Ersetzbarkeit und History-Constraints

- **Server-kontrolliert:** Wenn T eine Zustandsänderung von X nach Y verhindert, muss U dies auch verhindern. U darf stärker einschränken als T .
- **Client-kontrolliert:** Jede Aufrufreihenfolge, die T erlaubt, muss U auch erlauben.
 - $TraceSet(T) \subseteq TraceSet(U)$ (Die Menge der erlaubten Aufrufspuren von U ist eine Obermenge der Spuren von T).

Weitere Aspekte von Zusicherungen

- **Ausnahmebehandlung:** Eine Methode in einer Unterklasse darf nur Exceptions werfen, die der Aufrufer der Methode in der Oberklasse erwartet (dies ist Teil der Nachbedingung).
- **Genauigkeit:**
 - *Genau:* Große Abhängigkeit zwischen Client und Server.
 - *Ungenau:* Kleine Abhängigkeit (bevorzugt).
- **Faustregeln:** Zusicherungen sollten stabil, explizit, unmissverständlich sein und keine unnötigen Details festlegen.

Beziehungen zwischen Typen

Abstrakte Klassen und Interfaces

- **Abstrakte Klasse:** Eine Klasse, von der keine Objekte direkt erzeugt werden können (nur von ihren Unterklassen). Kann implementierte Methoden, abstrakte Methoden und Objektvariablen enthalten.
- **Interface:** Eine spezielle Form einer abstrakten Klasse in Java, die (traditionell) keine Objektvariablen und nur abstrakte Methoden (oder `default`/`static` Methoden) enthält.
- **Zweck:** Dienen als allgemeine Obertypen, um Ersetzbarkeit zu ermöglichen.
- **Richtlinie:** Interfaces sind Klassen vorzuziehen, außer Objektvariablen sind unvermeidbar.

Arten von Klassen-Beziehungen

1. Untertypbeziehung (Subtyping):

- **Konzept:** Objekt von T_2 ersetzt Objekt von T_1 .
- **Fokus:** Ersetzbarkeit (Substitutability).
- Code-Vererbung ist irrelevant.

2. Vererbungsbeziehung (Inheritance):

- **Konzept:** T_1 vereinfacht die Implementierung von T_2 .
- **Fokus:** Code-Wiederverwendung (Code Reuse).
- Ersetzbarkeit ist irrelevant.

3. Reale-Welt-Beziehung:

- **Konzept:** Begriff 2 *ist auch ein* Begriff 1 (z.B. "Dreieck ist ein Polygon").
- **Fokus:** Intuitives Verständnis im Entwurf.
- Wird oft zu einer Untertypbeziehung weiterentwickelt.

Untertypen versus Vererbung

- In Java erzwingt die Syntax oft eine Kopplung: Eine Untertypbeziehung setzt (meist) Vererbung voraus, und Vererbung impliziert eine (vom Compiler geprüfte) Untertypbeziehung.
- Die *wahre* Unterscheidung liegt in den **Zusicherungen**.
- **Vererbung** ist **direkte Codewiederverwendung** (leicht sichtbar).
- **Untertypbeziehung** ist **indirekte Codewiederverwendung** (schwer sichtbar, über Polymorphismus).
- **WICHTIG:** Indirekte Wiederverwendung (durch Ersetzbarkeit) ist langfristig viel wichtiger als direkte Wiederverwendung.

: [pp25v05, p.20](#)

Vererbung (Inheritance)

Direkte Codewiederverwendung

- **Vorteil:** Code muss nur einmal geschrieben und gewartet werden.
- **Nachteil:** Starke Abhängigkeit zwischen Ober- und Unterklasse.
- **Regel:** Nur von **stabilen** Klassen erben.

Techniken zur Implementierung von Vererbung

- **Verwendung von `super`:** Die Unterklasse kann die Methode der Oberklasse explizit aufrufen und deren Verhalten erweitern.
: [pp25v05, p.23|400](#)
- **Verhinderte Vererbung (Schlechtes Design):** Wenn die Methode in der Oberklasse monolithisch ist (ein großer Block), muss die Unterklasse sie komplett neu implementieren, selbst bei kleinen Änderungen.
: [pp25v05, p.24|400](#)
- **Vererbung durch Zerlegung (Template Method Pattern):** Die Oberklasse zerlegt den Algorithmus in mehrere (oft `protected`) Hilfsmethoden. Die Unterklasse überschreibt nur die kleinen, spezifischen Hilfsmethoden.
: [pp25v05, p.25|400](#)
- **Vererbung durch Parametrisierung:** Die Oberklasse implementiert die Logik in einer Hilfsmethode mit einem Parameter. Die Unterklasse ruft diese Hilfsmethode mit einem anderen Wert für den Parameter auf.
: [pp25v05, p.26|400](#)

Verdecken (Hiding) vs. Überschreiben (Overriding)

- **Variablen (Verdecken):**
 - Eine Variable in der Unterklasse *verdeckt* (hides) die Variable der Oberklasse mit gleichem Namen.
 - Beide Variablen existieren.
 - Zugriff auf Oberklassen-Variable: `super.var` oder `((Oberklasse)this).var`.
- **Methoden (Überschreiben):**

- Eine Methode in der Unterklasse *überschreibt* (overrides) die Methode der Oberklasse (gleiche Signatur).
- Zugriff auf Oberklassen-Methode: `super.method(...)`.
- Zugriff via `((Oberklasse)this).method(...)` ist **nicht** möglich (wegen dynamischem Binden wird trotzdem die Methode der Unterklasse aufgerufen).

Das **final** Schlüsselwort

- `public final method(...)`
 - Verhindert, dass die Methode in Unterklassen *überschrieben* wird.
 - Sollte eher vermieden werden.
- `public final class ...`
 - Verhindert, dass von der Klasse *geerbt* wird (es kann keine Unterklassen geben).
 - Nützlich in manchen Programmierstilen, um Ersetzbarkeit klar zu definieren (z.B. wenn alle konkreten Klassen `final` sind und nur von `abstract` Klassen oder Interfaces geerbt wird).

Steuerung der Sichtbarkeit

Geschachtelte Klassen (Nested Classes)

1. Statisch geschachtelte Klasse (Static Nested Class)

- Wird mit `static` deklariert.
- Gehört zur umschließenden **Klasse** (nicht zu einem Objekt).
- Kann *nur* auf `static` Member (Variablen/Methoden) der umschließenden Klasse zugreifen (auch `private static`).
- Instanziierung: `new EnclosingClass.StaticNestedClass()`

: pp25v05, p.29|400

2. Innere Klasse (Inner Class)

- Wird *ohne* `static` deklariert.
- Gehört zu einem **Objekt** (einer Instanz) der umschließenden Klasse.
- Kann auf *alle* Member (auch non-static und private) des umschließenden Objekts direkt zugreifen.
- Instanziierung: `enclosingObject.new InnerClass()`

: pp25v05, p.30|400

3. Anonyme innere Klasse (Anonymous Inner Class)

- Eine innere Klasse ohne Namen, die gleichzeitig definiert und instanziiert wird.
- Syntax: `new TypName() { ...Implementierung... }`
- Erzeugt ein Objekt eines *neuen Untertyps* von `TypName`.

4. Lambda-Ausdruck

- Eine verkürzte Syntax zur Erzeugung eines Objekts eines **Functional Interface**.

- Ein *Functional Interface* ist ein Interface mit *genau einer* abstrakten Methode.
- Beispiel: $Tx = (a, b) \rightarrow a + b$;
- Dies entspricht (vereinfacht) der anonymen Klasse:

```
new T() { int f(int a, int b) { return a + b; } }
```

Pakete (Packages)

- Ein Paket fasst alle Klassen im selben Ordner zusammen.
- Deklaration: `package paketName;`
- Vollqualifizierter Name: `myclasses.test.AClass.foo()`
- `import`-Deklarationen vereinfachen den Zugriff.

Sichtbarkeitsmodifikatoren

- **Lokal:** Sichtbar im selben Paket (auch außerhalb der definierenden Klasse).
- **Global:** Sichtbar auch außerhalb des Pakets.

: [pp25v05, p.33](#)

Anwendung der Sichtbarkeitskontrolle

- `public`: Für allgemein benötigte Methoden, Konstruktoren, Konstanten.
 - (Public Variablen sind verpönt -> Design-by-Contract).
- `private`: Für alles, was außerhalb der Klasse nicht benötigt wird.
 - **Ideal für Variablen** und Hilfsmethoden.
- `protected`: Für Member, die von Unterklassen benötigt werden, aber nicht zur öffentlichen API gehören.
 - (Sollte heute weitgehend vermieden werden).
- `(default)` (**Package-Private**): Nur für enge Zusammenarbeit von Klassen *im selben Paket*.
 - (Selten sinnvoll, oft falsch verwendet).

vo6

Dynamische Typinformation und statische Parametrisierung

Generizität

- **Was ist Generizität?**
 - Eine Form der **Parametrisierung**, bei der "Lücken" im Code (generische Parameter, meist **Typparameter**) erst bei der Verwendung durch konkrete Typen ersetzt werden.
 - Es ist ein **statischer Mechanismus**; die Typinformation wird zur Übersetzungszeit genutzt. Es findet kein dynamisches Binden wie bei Untertypen statt.
- **Wozu Generizität?**
 - **Code-Wiederverwendung**: Anstatt denselben Code für verschiedene Typen (z.B. eine `List` für `String` und eine `List` für `Integer`) mehrfach zu schreiben (Copy-Paste), schreibt man ihn nur einmal generisch (z.B. `List<T>`).
 - **Typsicherheit**: Der Compiler garantiert statisch, dass z.B. in eine `List<String>` nur `String`-Objekte eingefügt werden können. Ohne Generizität (z.B. `List` mit `Object`) würden Typfehler erst zur Laufzeit durch `ClassCastException` bemerkt.

Einfache Generizität in Java

- Generische Klassen oder Interfaces deklarieren Typparameter in spitzen Klammern (z.B. `<A>`).
- Innerhalb der Klasse/des Interface kann der Typparameter (z.B. `A`) wie ein normaler Referenztyp verwendet werden.
- **Wichtig**: Typparameter können in Java **nur durch Referenztypen** (Klassen, Interfaces) ersetzt werden, **nicht durch primitive Typen** (wie `int`, `boolean`).

Beispiel: Interfaces `Collection<A>` und `Iterator<A>`

: [pp25v06, p.3](#)

- `Collection<A>` definiert Methoden, die den Typparameter `A` verwenden (z.B. `void add(A elem)`).
- Bei Verwendung als `Collection<String>` ersetzt der Compiler `A` konzeptuell durch `String`.

Beispiel: Implementierung `List<A>`

: [pp25v06, p.4](#)

: [pp25v06, p.5](#)

- Die Klasse `List<A>` implementiert `Collection<A>`.
- Sie verwendet eine Hilfsklasse `Node<T>`. Der Typparameter `T` in `Node` ist unabhängig von `A` in `List`, wird aber bei der Verwendung (z.B. `private Node<A> head`) mit `A` gleichgesetzt.

- Konstruktoren (z.B. `new Node<A>(elem)`) geben selbst keine Typparameter an, aber bei der Objekterzeugung (`new`) muss der Typ angegeben werden.
- `ListIter` ist eine **innere Klasse** und kann direkt auf den Typparameter `A` der umschließenden `List`-Klasse zugreifen.

Beispiel: Verwendung von `List<A>`

[:pp25v06,p.6](#)

- `List<Integer> xs = new List<Integer>();`
- `xs.add(0);`
 - Hier funktioniert `0` (primitiver Typ `int`), weil Java **Autoboxing** durchführt (automatische Umwandlung von `int` zu `Integer`).
- `Integer x = xs.iterator().next();`
 - Das Ergebnis von `next()` ist dank Generizität `Integer`, kein `Object`. Es ist kein Cast nötig.
- `zs.add(ys);` // !! compiler shows error message !!
 - Der Compiler verhindert, dass eine `List<String>` (`ys`) in eine `List<List<Integer>>` (`zs`) eingefügt wird. Die Typsicherheit ist statisch garantiert.

Generische Methoden

- Auch einzelne Methoden können Typparameter deklarieren, unabhängig davon, ob die Klasse selbst generisch ist.
- Der Typparameter wird **vor** dem Rückgabetyt deklariert (z.B. `<A>`).

Beispiel: `Comparator<A>` und `max()`

[:pp25v06,p.7](#)

- `public interface Comparator<A>` definiert eine Schnittstelle zum Vergleichen von Objekten des Typs `A`.
- `public static <A> A max(Collection<A> xs, Comparator<A> c)`
 - `<A>` deklariert den Typparameter für diese Methode.
 - Die Methode findet das größte Element in einer `Collection<A>` gemäß einem `Comparator<A>`.
- **Typinferenz bei Methoden:**
 - Beim Aufruf `CollectionOps.max(xs, cx)` (wobei `xs` eine `List<Integer>` und `cx` ein `Comparator<Integer>` ist) muss `A` nicht explizit angegeben werden.
 - Der Compiler **leitet (inferiert)** aus den Argumenten (`xs` und `cx`) ab, dass `A` in diesem Aufruf `Integer` sein muss.

Gebundene Generizität (Bounded Generics)

- Oft reicht ein beliebiger Typ nicht aus. Man muss sicherstellen, dass der Typ bestimmte Methoden besitzt.
- **Schranken (Bounds):** Schränken ein, welche Typen einen Typparameter ersetzen dürfen.
- **Syntax:** `<T extends SomeClass & InterfaceA & InterfaceB>`
 - `extends` wird immer verwendet (auch für Interfaces).
 - Nur Typen, die **Untertypen aller Schranken** sind, sind erlaubt.
 - Im Rumpf der generischen Klasse/Methode kann auf alle `public` Methoden/Variablen der Schranken zugegriffen werden.
 - Ist keine Schranke angegeben (ungebunden), wird implizit `Object` als Schranke angenommen.

Beispiel: `Scene<T extends Scalable>`

[📖: pp25v06, p.9](#)

- `<T extends Scalable>`: Der Typparameter `T` ist auf Typen beschränkt, die `Scalable` implementieren.
- Dadurch ist der Compiler sicher, dass Objekte vom Typ `T` die Methode `scale()` besitzen, und erlaubt den Aufruf `e.scale(factor);`.

Rekursion in Schranken (F-gebundene Generizität):

[📖: pp25v06, p.11](#)

- `<A extends Comparable<A>>`: Dies ist eine häufige, rekursive Schranke.
- Sie bedeutet: "Erlaube jeden Typ `A`, der mit sich *selbst* vergleichbar ist" (z.B. `Integer implements Comparable<Integer>`).
- Dies ermöglicht der `max`-Methode, die `compareTo`-Methode auf den Elementen selbst aufzurufen (`w.compareTo(x)`).

Keine impliziten Untertypbeziehungen!

- **WICHTIG:** Generizität unterstützt **keine** impliziten Untertypbeziehungen.
- Nur weil `String` ein Untertyp von `Object` ist, ist `List<String>` **KEIN** Untertyp von `List<Object>`.

[📖: pp25v06, p.23](#)

- Arrays verletzen dieses Prinzip (sind *kovariant*): `Object[] ys = xs;` (wobei `xs` ein `String[]` ist) ist erlaubt.
 - Dies führt zu Laufzeitfehlern: `ys[0] = y;` (wobei `y` ein `Integer` ist) wirft eine `ArrayStoreException`.

[📖: pp25v06, p.23](#)

- Generics verhindern dies: `List<Object> ys = xs;` (wobei `xs` eine `List<String>` ist) erzeugt einen **Compile-Zeit-Fehler**.

Wildcards (flexible Schranken)

Um die strikte Invarianz (`List<String>` ist kein `List<Object>`) zu lockern, gibt es Wildcards.

- `? extends T` (**Kovarianz, Obergrenze**):

- "Ein unbekannter Typ, der *maximal* `T` ist (also `T` oder ein Untertyp)."
 - Wird für **Lesezugriffe** (lesende Position, z.B. Rückgabewerte) verwendet.
 - Man kann daraus lesen (man erhält garantiert ein `T`), aber man kann **nichts hineinschreiben** (außer `null`), da der Compiler den genauen Typ nicht kennt (es könnte eine `List<Triangle>` sein, in die man kein `Square` einfügen darf).
 - Beispiel: `void drawAll(List<? extends Polygon> p)` akzeptiert `List<Polygon>`, `List<Triangle>`, `List<Square>`.
 - **? super T (Kontravarianz, Untergrenze):**
 - "Ein unbekannter Typ, der *mindestens* `T` ist (also `T` oder ein Obertyp)."
 - Wird für **Schreibzugriffe** (schreibende Position, z.B. Eingangsparameter) verwendet.
 - Man kann hineinschreiben (ein `Square` passt immer in eine `List<? super Square>`, egal ob es `List<Square>`, `List<Polygon>` oder `List<Object>` ist), aber man kann **nichts typsicher lesen** (außer als `Object`).
 - Beispiel: `void addSquares(List<? super Square> to)`
 - **<?> (Ungebundenes Wildcard):**
 - Entspricht `<? extends Object>`. Man kann nur `Object` lesen und (fast) nichts schreiben.
-

Verwendung von Generizität

Richtlinien (Faustregeln)

- **Gleich strukturierte Klassen/Methoden:** Immer verwenden, wenn dieselbe Logik für verschiedene Typen gebraucht wird, z.B. **Containerklassen** (`List`, `Map`, `Set`).
- **Bibliotheken:** Klassen und Methoden in Bibliotheken sollten (fast) immer generisch sein.
- **Erkennen gleicher Strukturen:** Auch wenn es nicht wie ein Container aussieht: Wenn mehrere Variablen denselben (aber nicht von Anfang an fixen) Typ haben müssen (z.B. Konten für verschiedene, aber pro Konto konsistente Währungen), ist (gebundene) Generizität sinnvoll.
- **Abfangen erwarteter Änderungen:** Wenn erwartet wird, dass sich Parametertypen in Zukunft ändern (spezialisieren), kann Generizität helfen.
- **Generizität vs. Untertypen:** Sie ergänzen sich. Untertypen ermöglichen *heterogene* Listen (verschiedene Typen, ein gemeinsamer Obertyp). Generizität ermöglicht *homogene* Listen (alle Elemente vom exakt gleichen, aber austauschbaren Typ).
- **Laufzeiteffizienz:** Sollte **kein** Entscheidungsgrund für oder gegen Generizität sein (die Unterschiede sind meist minimal oder unerwartet).
- **Natürlichkeit:** Oft ist die "natürlichere" Lösung (basierend auf Erfahrung) die bessere.

Arten der Generizität (Übersetzung)

- **Homogene Übersetzung (Java, C#):**
 - Nennt sich **Type Erasure** (Typauslöschung).
 - Der Compiler übersetzt *eine* generische Klasse (z.B. `List<T>`) in *eine* nicht-generische `.class`-Datei (z.B. `List`).

- Alle Typparameter (`T`) werden im Bytecode durch ihre Schranke (oder `Object`, falls ungebunden) ersetzt.
- Der Compiler fügt **automatisch Typumwandlungen (Casts)** an den nötigen Stellen ein (z.B. `(String) list.get(0)`).
- **Heterogene Übersetzung (C++ Templates):**
 - Nennt sich **Code-Duplizierung**.
 - Der Compiler erzeugt für *jede* verwendete Typkombination (z.B. `List<String>`, `List<int>`) **neuen, separaten Maschinencode**.
 - **Vorteil:** Effizienter zur Laufzeit (keine Casts, keine Wrapper-Objekte für primitive Typen wie `int`).
 - **Nachteil:** Größere Programmdateien ("Code Bloat").

Typabfragen und Typumwandlungen

Verwendung dynamischer Typinformation

Zur Laufzeit kann auf Typinformationen zugegriffen werden.

- `obj.getClass()`: Gibt ein `Class`-Objekt zurück, das den **dynamischen Typ** des Objekts repräsentiert. Nützlich für Reflexion.
- `obj instanceof Typ`:
 - Prüft, ob der **dynamische Typ** von `obj` ein **Untertyp** von `Typ` ist.
 - Liefert `true`, wenn `obj` eine Instanz von `Typ` oder einer seiner Unterklassen ist.
 - Liefert `false`, wenn `obj` `null` ist oder kein Untertyp ist.
 - Beispiel: `if (p instanceof Student)`
- **Typumwandlung (Cast):** `(Typ)obj`
 - Teilt dem Compiler mit, dass `obj` (auch wenn nur als Obertyp deklariert) als `Typ` behandelt werden soll.
 - **Down-Cast:** (Vom Obertyp zum Untertyp, z.B. `(Student)person`)
 - Ist **riskant**.
 - Wird zur **Laufzeit** überprüft.
 - Schlägt fehl (wirft `ClassCastException`), wenn der dynamische Typ von `obj` kein Untertyp von `Typ` ist.
 - **Up-Cast:** (Vom Untertyp zum Obertyp, z.B. `(Person)student`)
 - Ist **immer sicher** und wird nicht zur Laufzeit geprüft.

Gefahren von Typabfragen und Casts

- **Umgehung der statischen Typsicherheit:** Sie schalten die Garantien des Compilers aus. Fehler (z.B. eine `ClassCastException`) passieren erst zur Laufzeit.
- **Schlechte Wartbarkeit:**
 - Der Code wird "fix verdrahtet" auf spezifische Typen.
 - `if (x instanceof T1) ... else if (x instanceof T2) ...`

- Wenn ein neuer Typ `T3` hinzukommt, müssen **alle** diese `instanceof`-Ketten im gesamten Programm gefunden und manuell erweitert werden.
- **Bessere Alternative: Dynamisches Binden** verwenden (eine Methode `doSomething()` definieren und in `T1`, `T2`, `T3` unterschiedlich implementieren; statt `if (x instanceof ...)` einfach `x.doSomething()` aufrufen).

Faustregel: Typabfragen und Typumwandlungen (Casts) sollen nach Möglichkeit **vermieden** werden.

Typumwandlungen und Generizität (Type Erasure)

- **Homogene Übersetzung (Type Erasure):**
 - `List<String>` wird im Bytecode zu `List` (dem "Raw Type").
 - `T` wird zu `Object` (oder der Schranke).
 - **Compiler fügt Casts ein:**
 - Ein generischer Aufruf `String y = xs.iterator().next();` (wobei `xs` eine `List<String>` ist)...
 - ...wird vom Compiler übersetzt in: `String y = (String) xs.iterator().next();`
 - Der Compiler garantiert (dank seiner statischen Prüfungen), dass dieser Cast zur Laufzeit **immer** erfolgreich sein wird (solange keine Raw Types verwendet werden).
 - **"Sichere" Typumwandlungen:**
 1. **Up-Casts:** (Immer sicher).
 2. Casts nach einer `instanceof`-Prüfung.
 3. Casts, die der Compiler bei Generizität einfügt (sind statisch geprüft).
 - **Gefahr: Raw Types (Typen ohne `<...>`):**
 - Wenn man `List` (Raw Type) statt `List<String>` verwendet, schaltet man die statischen Prüfungen der Generizität aus. Der Compiler fügt keine (oder falsche) Casts ein, was zu `ClassCastException`s führt.
 - **`instanceof` mit Generizität:**
 - `if (o instanceof List<String>)` // **COMPILE-FEHLER!**
 - **Warum?** Wegen Type Erasure. Zur Laufzeit gibt es keine Information über `<String>`. Das System kann eine `List<String>` nicht von einer `List<Integer>` unterscheiden.
 - **Erlaubt ist nur:** `if (o instanceof List)` (Prüfung auf den Raw Type).
-

Kovariante Probleme

- **Problem:** Situationen, in denen man sich (aus "natürlicher" Sicht) **kovariante Eingangsparameter** wünscht, obwohl die Theorie der Ersetzbarkeit **Kontravarianz** vorschreibt.
- **Beispiel (Tiere füttern):**
 - `class Animal { abstract void eat(Food x); }`

- `class Cow extends Animal { ... }`
- `class Grass extends Food { ... }`
- Man *möchte* `Cow.eat(Grass x)` überschreiben.
- Das ist aber **kovariant** zu `Animal.eat(Food x)` (da `Grass` ein Untertyp von `Food` ist).
- Dies ist **nicht erlaubt**, da es die Ersetzbarkeit verletzt (ein `Animal`-Client könnte versuchen, einer `Cow` `Meat` zu geben).
- **Lösung (Simulation mit `instanceof`):**
 - Die Methode in `Cow` behält die invariante Signatur `eat(Food x)`.
 - Innerhalb der Methode wird der dynamische Typ des Arguments geprüft.
 - Dies kann mit **Überladen** kombiniert werden, um Aufrufern, die den spezifischen Typ (`Grass`) kennen, einen direkten (effizienteren) Einstiegspunkt zu geben, während der allgemeine (polymorphe) Aufruf über `eat(Food x)` weiterhin funktioniert.